```
// ===== android-compose/app/src/main/java/com/xiaoling/MainActivity.kt =====
package com.xiaoling

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.lifecycle.viewmodel.compose.viewModel
import com.xiaoling.core.AppState
import com.xiaoling.service.AppForeground
import com.xiaoling.ui.XiaolingApp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val vm: AppState = viewModel()
            XiaolingApp(vm)
        }
    }

    // App ????,??????(? App ???);?????,?????????
    override fun onStart() { super.onStart(); AppForeground.active = true }
    override fun onStop() { super.onStop(); AppForeground.active = false }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/Account.kt =====
package com.xiaoling.core

import android.content.Context

/** ??????(?????????????) */
object Account {
    private const val PREF = "xiaoling_acct"

    fun isLoggedIn(ctx: Context) = token(ctx).isNotEmpty()
    fun token(ctx: Context) = sp(ctx).getString("token", "") ?: ""
    fun phone(ctx: Context) = sp(ctx).getString("phone", "") ?: ""
    fun userId(ctx: Context) = sp(ctx).getString("uid", "") ?: ""
    fun familyId(ctx: Context) = sp(ctx).getString("family_id", "") ?: ""
    fun membership(ctx: Context) = sp(ctx).getString("membership", "") ?: ""

    fun save(ctx: Context, token: String, phone: String, uid: String, familyId: String, membership: Str
        sp(ctx).edit()
            .putString("token", token).putString("phone", phone).putString("uid", uid)
            .putString("family_id", familyId).putString("membership", membership).apply()
    }

    fun setMembership(ctx: Context, tier: String) {
        sp(ctx).edit().putString("membership", tier).apply()
    }
```

```
    fun logout(ctx: Context) { sp(ctx).edit().clear().apply() }

    private fun sp(ctx: Context) = ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE)
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/ActionDispatcher.kt =====
package com.xiaoling.core

import android.content.Context
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.VibrationEffect
import android.os.Vibrator
import android.os.VibratorManager
import org.json.JSONObject

/**
 * ?????? action ????? Android ??(??/??/??/????)?
 * ????? ACTION_DIAL(???);SOS ? ACTION_CALL(? CALL_PHONE,???? DIAL)?
 */
object ActionDispatcher {

    /** @return ????????(???);?????? TTS ?? */
    fun execute(ctx: Context, action: JSONObject): String? {
        val app = ctx.applicationContext
        return when (action.optString("type")) {
            "CALL" -> {
                val target = action.optString("target")
                val num = Contacts.lookup(app, target)
                if (num.isNullOrBlank()) {
                    "???????$target?,??????????"
                } else {
                    view(app, Intent(Intent.ACTION_DIAL, Uri.parse("tel:$num")))
                    null
                }
            }
            "SOS" -> {
                val num = action.optString("call", "120")
                val call = Intent(Intent.ACTION_CALL, Uri.parse("tel:$num"))
                    .addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                try {
                    app.startActivity(call)
                } catch (e: Exception) {
                    // ?? CALL_PHONE ???:?????
                    view(app, Intent(Intent.ACTION_DIAL, Uri.parse("tel:$num")))
                }
                null
            }
```

```
            "CALL_NUMBER" -> {
                val num = action.optString("number")
                if (num.isNotBlank()) view(app, Intent(Intent.ACTION_DIAL, Uri.parse("tel:$num")))
                null
            }
            "OPEN_URI" -> {
                val uri = action.optString("uri")
                val ok = view(app, Intent(Intent.ACTION_VIEW, Uri.parse(uri)))
                if (!ok) {
                    // ???? ? ?????? geo:
                    val kw = Regex("keywords=([^&]+)").find(uri)?.groupValues?.get(1) ?: ""
                    view(app, Intent(Intent.ACTION_VIEW, Uri.parse("geo:0,0?q=$kw")))
                }
                null
            }
            "FRAUD_WARN" -> { vibrate(app); null }
            "PLAY", "REMIND", "SPEAK" -> null   // ?????(????? TTS ??)
            else -> null
        }
    }

    private fun view(ctx: Context, intent: Intent): Boolean {
        return try {
            ctx.startActivity(intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)); true
        } catch (e: Exception) { false }
    }

    private fun vibrate(ctx: Context) {
        try {
            val vib = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
                (ctx.getSystemService(Context.VIBRATOR_MANAGER_SERVICE) as VibratorManager).defaultVibr
            } else {
                @Suppress("DEPRECATION")
                ctx.getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
            }
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                vib.vibrate(VibrationEffect.createOneShot(500, VibrationEffect.DEFAULT_AMPLITUDE))
            } else {
                @Suppress("DEPRECATION") vib.vibrate(500)
            }
        } catch (e: Exception) { /* ????,?? */ }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/AlarmBus.kt =====
package com.xiaoling.core

import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.asSharedFlow
```

```
????????? V1.0   ? 4 ?
--------------------------------------------------------------------------------------------
/**
 * ???????:??(?????/????)??????,?? App UI ?????"??"?
 * App ???????????;App ??????? + ????? FraudStore.pending ???
 */
object AlarmBus {
    private val _events = MutableSharedFlow<String>(extraBufferCapacity = 8)
    val events = _events.asSharedFlow()
    fun post(reason: String) { _events.tryEmit(reason) }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/AppState.kt =====
package com.xiaoling.core

import android.app.Application
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.isActive
import kotlinx.coroutines.withTimeoutOrNull
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.launchIn
import kotlinx.coroutines.flow.onEach
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
import org.json.JSONObject

enum class Screen { Home, Settings, Login }

data class UiState(
    val caption: String = "?????,???????",
    val mascot: MascotState = MascotState.Idle,
    val listening: Boolean = false,
    val speaking: Boolean = false,
    val busy: Boolean = false,
    val screen: Screen = Screen.Home,
    val lastUser: String = "",
    val brainUrl: String = "",
    val live2d: Boolean = false,
    val membership: String = "",     // "" / "basic" / "premium"
    val loggedIn: Boolean = false,
    val phone: String = "",
    // ????????
    val fraudBlocked: Int = 0,
    val sosLabel: String = "?",
    val medsOk: Boolean = true,
    val familySynced: Boolean = false
)
```

```
?????????? V1.0   ? 5 ?
--------------------------------------------------------------------------------------------
/** ??:??????? ? ??(??/????) ? TTS ? ???? ? ???? */
class AppState(application: Application) : AndroidViewModel(application) {

    private val app = application
    private val _state = MutableStateFlow(
        UiState(
            brainUrl = Settings.brainUrl(app),
            fraudBlocked = FraudStore.count(app),
            live2d = Settings.live2dEnabled(app),
            membership = if (Account.isLoggedIn(app)) Account.membership(app) else Membership.tier(app)
            loggedIn = Account.isLoggedIn(app),
            phone = Account.phone(app)
        )
    )
    val state: StateFlow<UiState> = _state.asStateFlow()

    private val speech = SpeechController(app)
    private val tts = Tts(app) { id -> onSpeakDone(id) }

    @Volatile private var autoOn = false     // ????(????/ TTS ??????)
    @Volatile private var speaking = false   // TTS ???(????,??????)
    @Volatile private var alarmUntil = 0L    // ??????????(?????????)
    private var curUtt = ""                   // ?? TTS utteranceId(? flush ?????)

    /** ??????(?????,????) */
    private fun tierNow(): String = if (Account.isLoggedIn(app)) Account.membership(app) else Membershi
    private fun isPremium(): Boolean = tierNow() == "premium"

    init {
        // ??(??/??)????? ? ??????
        AlarmBus.events.onEach { onExternalFraud(it) }.launchIn(viewModelScope)
        // App ???????? / ????,??? 2 ?????????
        FraudStore.takePendingIfRecent(app)?.let { onExternalFraud(it) }
        // ???:???????(?????????????),??????
        subscribePush()
    }

    private fun subscribePush() {
        viewModelScope.launch {
            while (isActive) {
                if (isPremium()) PushClient.subscribe(app) { ev -> onFamilyEvent(ev) }   // ??????:????
                delay(3000)   // ??/????????
            }
        }
    }

    /** ?????????? ? ?????? */
    private fun onFamilyEvent(ev: org.json.JSONObject) {
        if (ev.optString("sender") == PushClient.deviceId(app)) return   // ????????,??????
        val text = ev.optString("text").ifBlank { "???????????" }
```

```
        val type = ev.optString("type")
        val prefix = when (type) {
            "fraud_call", "fraud_sms" -> "????????:"
            "sos" -> "?????????:"
            else -> "????:"
        }
        speaking = true
        _state.update { it.copy(caption = "?? $prefix$text", mascot = MascotState.Caring, speaking = tr
        curUtt = tts.speak(prefix + text)
    }

    /** ????/???????:????????,???+????? */
    private fun onExternalFraud(reason: String) {
        FraudStore.clearPending(app)
        val say = "??!$reason?????????????????????"
        speaking = true
        _state.update { it.copy(mascot = MascotState.Alarm, caption = say, busy = false, speaking = tru
        try { ActionDispatcher.execute(app, JSONObject().put("type", "FRAUD_WARN")) } catch (e: Excepti
        if (isPremium()) viewModelScope.launch { PushClient.emit(app, "fraud_call", reason, System.curr
        curUtt = tts.speak(say)
        scheduleAlarmReset()
    }

    private fun scheduleAlarmReset() {
        val until = System.currentTimeMillis() + 6500L
        alarmUntil = until
        viewModelScope.launch {
            delay(6600L)
            // ??????????? deadline ???????,???????????
            if (System.currentTimeMillis() >= alarmUntil) {
                _state.update {
                    if (it.mascot == MascotState.Alarm) it.copy(mascot = MascotState.Idle, caption = "?
                }
            }
        }
    }

    // ---------- ?????(???) ----------
    fun startAuto() {
        if (autoOn) return
        if (!speech.isAvailable) {
            _state.update { it.copy(caption = "???????????,????Google ?????") }
            return
        }
        autoOn = true
        listenOnce()
    }

    fun stopAuto() {
        autoOn = false
```

```
        speech.destroy()
        _state.update { it.copy(listening = false) }
    }

    private fun listenOnce() {
        if (!autoOn || speaking) return
        _state.update {
            it.copy(
                listening = true,
                caption = "?????????",
                mascot = if (it.mascot == MascotState.Alarm) it.mascot else MascotState.Listening
            )
        }
        speech.listen(
            onText = { t ->
                _state.update { it.copy(listening = false) }
                if (t.isNotBlank()) process(t) else restartSoon()
            },
            onError = { restartSoon() }
        )
    }

    private fun restartSoon() {
        _state.update { it.copy(listening = false) }
        if (!autoOn) return
        viewModelScope.launch { delay(400); if (autoOn && !speaking) listenOnce() }
    }

    // ---------- ??:???????(????),???????,?? ?2s ----------
    private fun process(text: String) {
        _state.update { it.copy(busy = true, mascot = MascotState.Thinking, caption = "???", lastUser =

        // ?? + ?????????,????(<0.3s ??)
        val local = LocalSafetyNet.handle(text) ?: LocalIntents.parse(text)
        if (local != null) { applyReply(local); return }

        viewModelScope.launch {
            // ????? 1.8s:whichever first???/???????,??? ?2s
            val reply = withTimeoutOrNull(1800) {
                try { BrainClient.ask(app, text) } catch (e: Exception) { null }
            } ?: Reply("??????????????????????????,?????????", null, "chat", 0.0)
            applyReply(reply)
        }
    }

    private fun applyReply(reply: Reply) {
        val type = reply.action?.optString("type")
        val hint = reply.action?.let { ActionDispatcher.execute(app, it) }
        val toSay = hint ?: reply.speech
        if (type == "FRAUD_WARN") FraudStore.inc(app)
```

```
        speaking = true
        _state.update {
            it.copy(
                busy = false,
                speaking = true,
                caption = toSay,
                mascot = stateFor(type, reply),
                fraudBlocked = FraudStore.count(app),
                sosLabel = if (type == "SOS") "??" else it.sosLabel
            )
        }
        curUtt = tts.speak(toSay)
        if (type == "FRAUD_WARN" || type == "SOS") {
            scheduleAlarmReset()
            if (isPremium()) {   // ???????:??????
                val pushType = if (type == "SOS") "sos" else "fraud_sms"
                viewModelScope.launch { PushClient.emit(app, pushType, reply.speech, System.currentTime
            }
        }
    }

    private fun stateFor(type: String?, reply: Reply): MascotState = when {
        type == "FRAUD_WARN" || reply.risk >= 0.55 || reply.skill.contains("??") -> MascotState.Alarm
        type == "SOS" || reply.skill.contains("??") -> MascotState.Alarm
        reply.skill == "chat" || reply.skill.contains("??") -> MascotState.Caring
        else -> MascotState.Talking
    }

    private fun onSpeakDone(id: String?) {
        if (id != curUtt) return   // ? flush ???????,??,??????/????
        speaking = false
        _state.update {
            val m = if (it.mascot == MascotState.Alarm) MascotState.Alarm else MascotState.Idle
            if (it.listening) it.copy(speaking = false) else it.copy(speaking = false, mascot = m)
        }
        if (autoOn) restartSoon()
    }

    // ---------- ?? / ??? ----------
    fun showScreen(s: Screen) {
        _state.update { it.copy(screen = s, fraudBlocked = FraudStore.count(app)) }
    }

    fun setBrainUrl(url: String) {
        Settings.setBrainUrl(app, url)
        _state.update { it.copy(brainUrl = Settings.brainUrl(app)) }
    }

    fun setLive2d(on: Boolean) {
        if (on && !isPremium()) {
```

```
?????????? V1.0   ? 9 ?
-------------------------------------------------------------------------------------------
            _state.update { it.copy(caption = "3D ???????????,??????") }
            return
        }
        Settings.setLive2d(app, on)
        _state.update { it.copy(live2d = on) }
    }

    /** ????:???(??)?????????(??????,????)? */
    fun buyPlan(plan: String, method: String) {
        _state.update { it.copy(caption = "????$method ???") }
        viewModelScope.launch {
            val ok = PayClient.pay(app, plan, method, Account.phone(app))
            if (ok) {
                if (Account.isLoggedIn(app)) Account.setMembership(app, plan) else Membership.set(app,
                val label = Membership.label(plan)
                _state.update { it.copy(membership = plan, caption = "$label ????,????!") }
                curUtt = tts.speak("$label ????,???????")
            } else {
                _state.update { it.copy(caption = "?????,??????") }
            }
        }
    }

    // ---------- ??(?????) ----------
    fun sendCode(phone: String) {
        viewModelScope.launch {
            val ok = AuthClient.sendCode(app, phone)
            _state.update { it.copy(caption = if (ok) "??????(????? 1234)" else "???????,?????/???") }
        }
    }

    fun login(phone: String, code: String) {
        _state.update { it.copy(caption = "?????") }
        viewModelScope.launch {
            val r = AuthClient.login(app, phone, code)
            if (r != null && r.optBoolean("ok", true) && r.has("token")) {
                val member = r.optString("membership", "")
                Account.save(app, r.optString("token"), phone, r.optString("uid"), r.optString("family_
                _state.update {
                    it.copy(loggedIn = true, phone = phone, membership = member,
                        screen = Screen.Settings, caption = "????,????!")
                }
            } else {
                val msg = r?.optString("msg", "") ?: ""
                _state.update { it.copy(caption = if (msg.isNotBlank()) msg else "????,???????? 1234")
            }
        }
    }

    fun logout() {
```

```
????????? V1.0   ? 10 ?
--------------------------------------------------------------------------------------------
        Account.logout(app)
        _state.update { it.copy(loggedIn = false, phone = "", membership = Membership.tier(app), live2d
    }

    /**
     * ??????(????)?????:
     *  ? ??????????? AppID + ??????;
     *  ? ???? OpenSDK,IWXAPI.sendReq(SendAuth.Req) ??????,WXEntryActivity ? code;
     *  ? ? code ??? /auth/wx_login,?? code2session ? openid + ??,?????
     * ??????? /auth/wx_login(demo code)???????
     */
    fun wxLogin() {
        _state.update { it.copy(caption = "?????????") }
        viewModelScope.launch {
            val r = AuthClient.wxLogin(app)
            if (r != null && r.has("token")) {
                val member = r.optString("membership", "")
                val ph = r.optString("phone", "????")
                Account.save(app, r.optString("token"), ph, r.optString("uid"), r.optString("family_id"
                _state.update { it.copy(loggedIn = true, phone = ph, membership = member, screen = Scre
            } else {
                _state.update { it.copy(caption = "???????(???????)?") }
            }
        }
    }

    fun syncFamily() {
        if (!isPremium()) {
            _state.update { it.copy(caption = "??????????,?????????????") }
            return
        }
        viewModelScope.launch {
            val msg = try {
                val ctx = JSONObject().put("scene", "family_sync")
                BrainClient.ask(app, "????????????", ctx).speech
            } catch (e: Exception) {
                "??????,????????????"
            }
            _state.update { it.copy(familySynced = true, caption = msg) }
            curUtt = tts.speak(msg)
        }
    }

    override fun onCleared() {
        super.onCleared()
        speech.destroy()
        tts.shutdown()
    }
}
```

```
????????? V1.0   ? 11 ?
--------------------------------------------------------------------------------
// ===== android-compose/app/src/main/java/com/xiaoling/core/AuthClient.kt =====
package com.xiaoling.core

import android.content.Context
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.net.HttpURLConnection
import java.net.URL

/**
 * ?????(???)?????:
 *  - sendCode:??? /auth/send_code(?????????;demo ??????????)?
 *  - login:??? /auth/login(phone+code)? {token,uid,family_id,membership}?
 *  ????:???????(???/???)? OTP + ??;token ? JWT/OAuth?demo ????? 1234?
 */
object AuthClient {

    suspend fun sendCode(ctx: Context, phone: String): Boolean = withContext(Dispatchers.IO) {
        post(ctx, "/auth/send_code", JSONObject().put("phone", phone)) != null
    }

    /** @return ???????? JSON(token/uid/family_id/membership),?? null */
    suspend fun login(ctx: Context, phone: String, code: String): JSONObject? = withContext(Dispatchers
        post(ctx, "/auth/login", JSONObject().put("phone", phone).put("code", code))
    }

    /** ??????:????????? code;demo ??? code? */
    suspend fun wxLogin(ctx: Context): JSONObject? = withContext(Dispatchers.IO) {
        post(ctx, "/auth/wx_login", JSONObject().put("code", "demo-wx-code"))
    }

    private fun post(ctx: Context, path: String, body: JSONObject): JSONObject? {
        return try {
            val url = URL(Settings.brainUrl(ctx).trimEnd('/') + path)
            val c = (url.openConnection() as HttpURLConnection).apply {
                requestMethod = "POST"; doOutput = true
                connectTimeout = 3000; readTimeout = 3500
                setRequestProperty("Content-Type", "application/json; charset=utf-8")
            }
            c.outputStream.use { it.write(body.toString().toByteArray(Charsets.UTF_8)) }
            val ok = c.responseCode in 200..299
            val resp = (if (ok) c.inputStream else c.errorStream)?.bufferedReader()?.use { it.readText(
            c.disconnect()
            if (ok) JSONObject(resp).takeIf { it.optBoolean("ok", true) } else null
        } catch (e: Exception) { null }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/BrainClient.kt =====
```

```
????????? V1.0   ? 12 ?
--------------------------------------------------------------------------------
package com.xiaoling.core

import android.content.Context
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.io.IOException
import java.net.HttpURLConnection
import java.net.URL

/** ???????:POST /dialogue,??? HttpURLConnection + org.json,????? */
object BrainClient {

    /** ? IO ?????;?????,?????? LocalSafetyNet */
    suspend fun ask(ctx: Context, text: String, context: JSONObject? = null): Reply =
        withContext(Dispatchers.IO) {
            val body = JSONObject()
                .put("user_id", "u001")
                .put("text", text)
                .apply { if (context != null) put("context", context) }
                .toString()

            val url = URL(Settings.brainUrl(ctx).trimEnd('/') + "/dialogue")
            val c = (url.openConnection() as HttpURLConnection).apply {
                requestMethod = "POST"
                doOutput = true
                connectTimeout = 1200
                readTimeout = 1600
                setRequestProperty("Content-Type", "application/json; charset=utf-8")
            }
            try {
                c.outputStream.use { it.write(body.toByteArray(Charsets.UTF_8)) }
                val code = c.responseCode
                val stream = if (code in 200..299) c.inputStream else c.errorStream
                val resp = stream?.bufferedReader(Charsets.UTF_8)?.use { it.readText() } ?: ""
                if (code !in 200..299) throw IOException("HTTP $code")
                val j = JSONObject(resp)
                Reply(
                    j.optString("speech", "????"),
                    j.optJSONObject("action"),
                    j.optString("skill", ""),
                    j.optDouble("risk", 0.0)
                )
            } finally {
                c.disconnect()
            }
        }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/Contacts.kt =====
```

```
????????? V1.0   ? 13 ?
--------------------------------------------------------------------------------
package com.xiaoling.core

import android.content.Context
import android.provider.ContactsContract

/** ???:?? ? ??(? READ_CONTACTS ??) */
object Contacts {
    fun lookup(ctx: Context, name: String): String? {
        if (name.isBlank()) return null
        val uri = ContactsContract.CommonDataKinds.Phone.CONTENT_URI
        val projection = arrayOf(ContactsContract.CommonDataKinds.Phone.NUMBER)
        val selection = ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME + " LIKE ?"
        val args = arrayOf("%$name%")
        return try {
            ctx.contentResolver.query(uri, projection, selection, args, null)?.use { cur ->
                if (cur.moveToFirst()) cur.getString(0)?.replace(" ", "")?.replace("-", "") else null
            }
        } catch (e: Exception) {
            null
        }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/FraudStore.kt =====
package com.xiaoling.core

import android.content.Context

/** ???????? + ???????(? App ??????????) */
object FraudStore {
    private const val PREF = "xiaoling"
    private const val KEY = "call_fraud_blocked"
    private const val KEY_PENDING = "pending_fraud"
    private const val KEY_PENDING_AT = "pending_fraud_at"

    fun inc(ctx: Context): Int {
        val sp = ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE)
        val n = sp.getInt(KEY, 0) + 1
        sp.edit().putInt(KEY, n).apply()
        return n
    }

    fun count(ctx: Context): Int =
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE).getInt(KEY, 0)

    fun setPending(ctx: Context, reason: String) {
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE).edit()
            .putString(KEY_PENDING, reason)
            .putLong(KEY_PENDING_AT, System.currentTimeMillis())
            .apply()
```

```
    }

    /** ?? 2 ?????????,?? null */
    fun takePendingIfRecent(ctx: Context): String? {
        val sp = ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE)
        val reason = sp.getString(KEY_PENDING, null) ?: return null
        val at = sp.getLong(KEY_PENDING_AT, 0)
        clearPending(ctx)
        return if (System.currentTimeMillis() - at <= 120_000) reason else null
    }

    fun clearPending(ctx: Context) {
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE).edit()
            .remove(KEY_PENDING).remove(KEY_PENDING_AT).apply()
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/FraudText.kt =====
package com.xiaoling.core

/**
 * ??/??????(??,?? server/fraud_rules.json ??? + ???)?
 * ?? (????, ??)?
 */
object FraudText {
    private val redline = listOf(
        "????", "????", "?????????", "???", "??????",
        "????", "???????", "????", "????", "????"
    )
    private val words = listOf(
        "???", "????", "???", "????", "??", "?????", "???",
        "???", "?????", "????", "?????", "??", "??", "??",
        "??", "??", "??", "???", "????????", "?????"
    )

    fun assess(text: String): Pair<Boolean, String> {
        val t = text
        val red = redline.firstOrNull { it in t }
        if (red != null) return true to "?????$red?,???????"
        val hits = words.filter { it in t }
        if (hits.size >= 2) return true to ("???" + hits.take(2).joinToString("?") + "?")
        if (hits.size == 1) return true to "???${hits[0]}?,?????"
        return false to ""
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/LocalIntents.kt =====
package com.xiaoling.core

import org.json.JSONObject
```

```
????????? V1.0   ? 15 ?
------------------------------------------------------------------------------------------

/**
 * ?????:????(???/??/??/?/??/????)??????,???? ? ?????
 * ?? server/skills.py;?????? null,?????????/???????? <0.3s ???
 */
object LocalIntents {

    // ??????(?/?/?),??????
    private val PHRASES: Map<String, Pair<String, String>> = mapOf(  // ??? -> (??, ??)
        "??" to ("Hello" to "??"),
        "??" to ("Thank you" to "??"),
        "??" to ("Goodbye" to "??"),
        "???" to ("How much is it?" to "???"),
        "?????" to ("Where is the toilet?" to "?????"),
        "????" to ("I don’t feel well" to "????"),
        "???" to ("Please help me" to "???"),
        "?????" to ("I need to go to the hospital" to "?????"),
        "????" to ("Call an ambulance" to "????"),
        "????" to ("I don’t understand" to "????"),
        "?????" to ("Please speak slower" to "????"),
        "????" to ("Have you eaten?" to "????"),
        "???" to ("Good morning" to "??"),
        "??" to ("Good night" to "??"),
        "???" to ("Take care" to "??"),
        "????" to ("What time is it now?" to "????"),
        "???" to ("That’s too expensive" to "???"),
        "???" to ("I love you" to "???"),
        "????" to ("Wish you good health" to "??????")
    )

    fun parse(text: String): Reply? {
        // ?? ????(??????)??
        parseTranslate(text)?.let { return it }

        var m = Regex("(?:?(?:?)?????|??|?????)\\s*(.+)").find(text)
        if (m != null) {
            val name = clean(m.groupValues[1])
            return Reply("??,?????$name ????",
                JSONObject().put("type", "CALL").put("target", name), "???", 0.0)
        }

        m = Regex("(?:????|?|?????|???)\\s*(.+)").find(text)
        if (m != null) {
            val dest = clean(m.groupValues[1].replace(Regex("(???|???)$"), ""))
            if (dest.isNotBlank()) {
                val uri = "androidamap://poi?sourceApplication=xiaoling&keywords=$dest&dev=0"
                return Reply("??,??????,???$dest?",
                    JSONObject().put("type", "OPEN_URI").put("uri", uri), "??", 0.0)
            }
        }
```

```
        if (Regex("????.*(??|???|??|??|??)").containsMatchIn(text)) {
            return Reply("??,???????,???",
                JSONObject().put("type", "REMIND").put("raw", text), "????", 0.0)
        }

        if (Regex("(?|?|??|?????|?).*(?|?|?|?|??|??|??)").containsMatchIn(text)) {
            val kw = clean(Regex("(?:?|?|??|?????|?)\\s*(.+)").find(text)?.groupValues?.get(1) ?: "")
            return Reply("??,?????$kw?",
                JSONObject().put("type", "PLAY").put("keyword", kw), "????", 0.0)
        }

        // ?? ????????(?????,?????)??
        if (Regex("^(??|??|??|???|???|???|?|??)").containsMatchIn(text)) {
            return Reply("???,??,??????????????", null, "chat", 0.0)
        }

        return null
    }

    private fun parseTranslate(text: String): Reply? {
        if (!Regex("??|???|?.{0,3}?|??|??").containsMatchIn(text)) return null
        val lang = when {
            Regex("??|???|??").containsMatchIn(text) -> "cantonese"
            Regex("???|??").containsMatchIn(text) -> "mandarin"
            else -> "english"
        }
        val langCn = if (lang == "cantonese") "??" else if (lang == "mandarin") "???" else "??"
        val content = clean(text.replace(
            Regex("(?)?(??)?(?)?|??(?|?)?|?.{0,3}?(???|?)?|???|??|??|??|??|???|??|???|??|??|[,??!]")
        if (content.isBlank()) return null
        val hit = PHRASES.entries.firstOrNull { content.contains(it.key) || it.key.contains(content) }?
        val out = when {
            lang == "mandarin" -> content
            hit != null && lang == "english" -> hit.first
            hit != null -> hit.second
            else -> null
        }
        // ????? ? ?? null,?????????(???);??????????
        if (out == null) return null
        return Reply("$content ?$langCn ?:$out",
            JSONObject().put("type", "SPEAK").put("text", out).put("lang", lang), "????", 0.0)
    }

    private fun clean(s: String): String =
        s.trim().replace(Regex("[????,?!\\s]+$"), "")
}


// ===== android-compose/app/src/main/java/com/xiaoling/core/LocalSafetyNet.kt =====
```

```
????????? V1.0   ? 17 ?
--------------------------------------------------------------------------------------------
package com.xiaoling.core

import org.json.JSONObject

/**
 * ??????:??/???????,??????????????????
 * ????? server/fraud_rules.json ? redline ? skills ? SOS ???
 */
object LocalSafetyNet {
    private val sos = Regex("??|??|????|???|??|???|??|??|120")
    private val redline = listOf(
        "????", "????", "?????????", "???", "??????",
        "????", "???????", "????", "???", "????"
    )

    fun handle(text: String): Reply? {
        if (sos.containsMatchIn(text)) {
            val a = JSONObject()
                .put("type", "SOS").put("call", "120")
                .put("notify_family", true).put("send_location", true)
            return Reply("??,???????120,??????", a, "????", 1.0)
        }
        val hit = redline.firstOrNull { it in text }
        if (hit != null) {
            val a = JSONObject().put("type", "FRAUD_WARN")
                .put("level", "high").put("category", "????")
            return Reply(
                "??!??????:?????$hit???????????????!",
                a, "?????", 0.96
            )
        }
        return null
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/MascotState.kt =====
package com.xiaoling.core

/** ??????,?? Avatar ???/??/?? */
enum class MascotState { Idle, Listening, Thinking, Talking, Alarm, Caring }

// ===== android-compose/app/src/main/java/com/xiaoling/core/Membership.kt =====
package com.xiaoling.core

import android.content.Context

/** ????(?????)?tier: "" ??? / "basic" ?? / "premium" ?? */
object Membership {
    private const val PREF = "xiaoling"
    private const val KEY = "member_tier"
```

```
?????????? V1.0   ? 18 ?
------------------------------------------------------------------------------------------

    fun tier(ctx: Context): String =
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE).getString(KEY, "") ?: ""

    fun set(ctx: Context, tier: String) {
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE).edit().putString(KEY, tier).apply()
    }

    fun label(tier: String): String = when (tier) {
        "basic" -> "????"
        "premium" -> "????"
        else -> "???"
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/Notifier.kt =====
package com.xiaoling.core

import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.Context
import android.os.Build
import androidx.core.app.NotificationCompat

/** ??????????(??/????) */
object Notifier {
    private const val CH = "fraud_alert"

    fun warn(ctx: Context, title: String, text: String, id: Int) {
        val nm = ctx.getSystemService(Context.NOTIFICATION_SERVICE) as? NotificationManager ?: return
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            nm.createNotificationChannel(
                NotificationChannel(CH, "????", NotificationManager.IMPORTANCE_HIGH)
            )
        }
        val n = NotificationCompat.Builder(ctx, CH)
            .setSmallIcon(android.R.drawable.stat_sys_warning)
            .setContentTitle(title)
            .setContentText(text)
            .setStyle(NotificationCompat.BigTextStyle().bigText(text))
            .setPriority(NotificationCompat.PRIORITY_HIGH)
            .setAutoCancel(true)
            .build()
        try { nm.notify(id, n) } catch (e: SecurityException) { /* ??????,?? */ }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/NumberReputation.kt =====
package com.xiaoling.core
```

```
????????? V1.0   ? 19 ?
--------------------------------------------------------------------------------------------
/**
 * ??????(??)??? level ? {safe, warn, block} + ???
 * ??:??? App ??????????,??????????????;
 * ???????????? / ??????
 */
object NumberReputation {
    // ?????(???? / ????)
    private val blacklist = setOf<String>()

    fun assess(raw: String): Pair<String, String> {
        val c = raw.replace(Regex("[^0-9+]"), "")
        if (c.isBlank()) return "safe" to ""
        if (c in blacklist) return "block" to "??????"
        if (suspicious(c)) return "warn" to reason(c)
        return "safe" to ""
    }

    private fun suspicious(c: String): Boolean {
        if (c.startsWith("+") && !c.startsWith("+86")) return true
        if (c.startsWith("00") || c.startsWith("95") || c.startsWith("96") ||
            c.startsWith("170") || c.startsWith("171")
        ) return true
        val d = c.removePrefix("+")
        return d.length < 7 || d.length > 12
    }

    private fun reason(c: String): String = when {
        c.startsWith("+") && !c.startsWith("+86") -> "????"
        c.startsWith("170") || c.startsWith("171") -> "???????"
        c.startsWith("00") -> "????/??"
        c.startsWith("95") || c.startsWith("96") -> "????(??????)"
        else -> "????"
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/PayClient.kt =====
package com.xiaoling.core

import android.content.Context
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.net.HttpURLConnection
import java.net.URL

/**
 * ?????(?? ? ????? ? ?? ? ??)?
 * ???????:??? /pay/create ??;????/??????????? SDK + ??? + ?????
 * @return ??????
 */
```

```
????????? V1.0   ? 20 ?
--------------------------------------------------------------------------------
object PayClient {

    suspend fun pay(ctx: Context, plan: String, method: String, phone: String = ""): Boolean = withCont
        val orderId = createOrder(ctx, plan, method, phone)   // ????(?????/????)
        // ==== ???????? ====
        // ??:IWXAPI.sendReq(PayReq) ?????? prepayId ??????????;
        // ???:PayTask(activity).payV2(orderInfo) ?????;
        // ???????????? /pay/notify ?????,?? demo ????????
        orderId != null || true   // demo:???????????,????????
    }

    private fun createOrder(ctx: Context, plan: String, method: String, phone: String): String? {
        return try {
            val body = JSONObject().put("plan", plan).put("method", method).put("phone", phone).toStrin
            val url = URL(Settings.brainUrl(ctx).trimEnd('/') + "/pay/create")
            val c = (url.openConnection() as HttpURLConnection).apply {
                requestMethod = "POST"; doOutput = true
                connectTimeout = 2500; readTimeout = 3000
                setRequestProperty("Content-Type", "application/json; charset=utf-8")
            }
            c.outputStream.use { it.write(body.toByteArray(Charsets.UTF_8)) }
            val ok = c.responseCode in 200..299
            val resp = (if (ok) c.inputStream else c.errorStream)?.bufferedReader()?.use { it.readText(
            c.disconnect()
            if (ok) JSONObject(resp).optString("orderId", null) else null
        } catch (e: Exception) {
            null
        }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/PorcupineWakeEngine.kt =====
package com.xiaoling.core

import android.content.Context
import java.io.File

/**
 * Picovoice Porcupine ??????(???)?
 * ????? SDK,?????????/????;????????,? WakeService ????????
 * ????? ? ?? onWake()?
 */
class PorcupineWakeEngine(private val ctx: Context) {

    private var manager: Any? = null

    val isAvailable: Boolean get() = WakeConfig.available(ctx)

    /** ??????;???? true???(? key/??/SDK)?? false ?????? */
    fun start(onWake: () -> Unit): Boolean {
```

```
        if (!isAvailable) return false
        return try {
            val keywordPath = copyAsset(WakeConfig.KEYWORD_ASSET)
            val modelPath = copyAsset(WakeConfig.MODEL_ASSET)

            // ???? PorcupineManager.Builder,???????
            val builderCls = Class.forName("ai.picovoice.porcupine.PorcupineManager\$Builder")
            val callbackCls = Class.forName("ai.picovoice.porcupine.PorcupineManagerCallback")
            val builder = builderCls.getConstructor().newInstance()

            builderCls.getMethod("setAccessKey", String::class.java).invoke(builder, WakeConfig.accessK
            builderCls.getMethod("setKeywordPath", String::class.java).invoke(builder, keywordPath)
            builderCls.getMethod("setModelPath", String::class.java).invoke(builder, modelPath)

            val callback = java.lang.reflect.Proxy.newProxyInstance(
                callbackCls.classLoader, arrayOf(callbackCls)
            ) { _, method, _ ->
                if (method.name == "invoke") onWake()
                null
            }
            manager = builderCls.getMethod("build", Context::class.java, callbackCls)
                .invoke(builder, ctx, callback)
            manager?.javaClass?.getMethod("start")?.invoke(manager)
            true
        } catch (e: Throwable) {
            stop()
            false
        }
    }

    fun stop() {
        try {
            manager?.let {
                it.javaClass.getMethod("stop").invoke(it)
                it.javaClass.getMethod("delete").invoke(it)
            }
        } catch (e: Throwable) { /* ?? */ }
        manager = null
    }

    /** Porcupine ??????,? assets ?????? */
    private fun copyAsset(name: String): String {
        val out = File(ctx.filesDir, name.substringAfterLast('/'))
        if (!out.exists() || out.length() == 0L) {
            ctx.assets.open(name).use { input -> out.outputStream().use { input.copyTo(it) } }
        }
        return out.absolutePath
    }
}
```

????????? V1.0   ? 22 ?

```
// ===== android-compose/app/src/main/java/com/xiaoling/core/PushClient.kt =====
package com.xiaoling.core

import android.content.Context
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.isActive
import kotlinx.coroutines.job
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.net.HttpURLConnection
import java.net.URL
import java.util.UUID
import kotlin.coroutines.coroutineContext

/**
 * ???????(????)?
 *  - emit:???????????,?????????;
 *  - subscribe:????? SSE ???????(App ????)?
 *  ???????? App????????(??/??/??/??/APNs)?? ? emit ????
 */
object PushClient {

    private const val DEFAULT_FAMILY = "family-100086"
    private const val PREF = "xiaoling"
    private const val KEY_DEVICE = "device_id"

    fun familyId(ctx: Context): String = Account.familyId(ctx).ifBlank { DEFAULT_FAMILY }   // ????????

    fun deviceId(ctx: Context): String {
        val sp = ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE)
        var id = sp.getString(KEY_DEVICE, null)
        if (id.isNullOrBlank()) { id = UUID.randomUUID().toString(); sp.edit().putString(KEY_DEVICE, id
        return id
    }

    /** ????(????)?best-effort,????????? */
    suspend fun emit(ctx: Context, type: String, text: String, atMs: Long) = withContext(Dispatchers.IO
        try {
            val body = JSONObject()
                .put("family_id", familyId(ctx)).put("sender", deviceId(ctx))
                .put("type", type).put("text", text).put("at", atMs / 1000).toString()
            val url = URL(Settings.brainUrl(ctx).trimEnd('/') + "/push/emit")
            val c = (url.openConnection() as HttpURLConnection).apply {
                requestMethod = "POST"; doOutput = true
                connectTimeout = 2500; readTimeout = 2500
                setRequestProperty("Content-Type", "application/json; charset=utf-8")
            }
            c.outputStream.use { it.write(body.toByteArray(Charsets.UTF_8)) }
            c.responseCode
            c.disconnect()
```

```
????????? V1.0   ? 23 ?
--------------------------------------------------------------------------------------------
            // ==== ????????:???? ??/?? ??? API,???????? ====
        } catch (e: Exception) { /* ?? */ }
    }

    /** ???????(?????),SSE ???;???????????????? */
    suspend fun subscribe(ctx: Context, onEvent: (JSONObject) -> Unit) = withContext(Dispatchers.IO) {
        var conn: HttpURLConnection? = null
        // ?????(? ViewModel ??)?????,???? readLine() ????,????/socket ??
        val handle = coroutineContext.job.invokeOnCompletion { runCatching { conn?.disconnect() } }
        try {
            val fid = familyId(ctx)
            val url = URL(Settings.brainUrl(ctx).trimEnd('/') + "/push/subscribe?family_id=$fid")
            conn = (url.openConnection() as HttpURLConnection).apply {
                requestMethod = "GET"
                connectTimeout = 4000; readTimeout = 0
                setRequestProperty("Accept", "text/event-stream")
            }
            conn.inputStream.bufferedReader(Charsets.UTF_8).use { reader ->
                while (coroutineContext.isActive) {
                    val line = reader.readLine() ?: break
                    if (line.startsWith("data:")) {
                        val json = line.removePrefix("data:").trim()
                        if (json.isNotEmpty() && json != "{}") {
                            try { onEvent(JSONObject(json)) } catch (e: Exception) {}
                        }
                    }
                }
            }
        } catch (e: Exception) {
            /* ??????? */
        } finally {
            handle.dispose()
            runCatching { conn?.disconnect() }
        }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/Reply.kt =====
package com.xiaoling.core

import org.json.JSONObject

/** ????:??? + ?????? + ???? + ??? */
data class Reply(
    val speech: String,
    val action: JSONObject?,
    val skill: String,
    val risk: Double
)
```

```
????????? V1.0   ? 24 ?
--------------------------------------------------------------------------------
// ===== android-compose/app/src/main/java/com/xiaoling/core/Settings.kt =====
package com.xiaoling.core

import android.content.Context
import com.xiaoling.BuildConfig

/** ?????:????? App ???,?? BuildConfig ??? */
object Settings {
    private const val PREF = "xiaoling"
    private const val KEY_URL = "brain_url"

    fun brainUrl(ctx: Context): String {
        val sp = ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE)
        val v = sp.getString(KEY_URL, null)
        return if (v.isNullOrBlank()) BuildConfig.BRAIN_URL else v
    }

    fun setBrainUrl(ctx: Context, url: String) {
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE)
            .edit().putString(KEY_URL, url.trim()).apply()
    }

    private const val KEY_LIVE2D = "live2d_enabled"
    fun live2dEnabled(ctx: Context): Boolean =
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE).getBoolean(KEY_LIVE2D, false)

    fun setLive2d(ctx: Context, on: Boolean) {
        ctx.getSharedPreferences(PREF, Context.MODE_PRIVATE).edit().putBoolean(KEY_LIVE2D, on).apply()
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/SpeechController.kt =====
package com.xiaoling.core

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.speech.RecognitionListener
import android.speech.RecognizerIntent
import android.speech.SpeechRecognizer

/**
 * ????????(??)???:???????????;RECORD_AUDIO ?????
 * ?? RecognitionListener ?????????
 */
class SpeechController(private val ctx: Context) {

    private var recognizer: SpeechRecognizer? = null

    val isAvailable: Boolean get() = SpeechRecognizer.isRecognitionAvailable(ctx)
```

```
    fun listen(onText: (String) -> Unit, onError: (Int) -> Unit) {
        if (!isAvailable) { onError(-1); return }
        val textCb = onText          // ??,??? override ??????(????)
        val errCb = onError
        recognizer?.destroy()
        recognizer = SpeechRecognizer.createSpeechRecognizer(ctx).also { r ->
            r.setRecognitionListener(object : RecognitionListener {
                override fun onResults(results: Bundle) {
                    val text = results
                        .getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION)
                        ?.firstOrNull().orEmpty()
                    textCb(text)
                }
                override fun onError(error: Int) { errCb(error) }
                override fun onReadyForSpeech(params: Bundle?) {}
                override fun onBeginningOfSpeech() {}
                override fun onRmsChanged(rmsdB: Float) {}
                override fun onBufferReceived(buffer: ByteArray?) {}
                override fun onEndOfSpeech() {}
                override fun onPartialResults(partialResults: Bundle?) {}
                override fun onEvent(eventType: Int, params: Bundle?) {}
            })
        }
        val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
            putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL, RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
            putExtra(RecognizerIntent.EXTRA_LANGUAGE, "zh-CN")
            putExtra(RecognizerIntent.EXTRA_MAX_RESULTS, 1)
            // ????:???? ~0.8s ???,?????
            putExtra(RecognizerIntent.EXTRA_SPEECH_INPUT_COMPLETE_SILENCE_LENGTH_MILLIS, 800)
            putExtra(RecognizerIntent.EXTRA_SPEECH_INPUT_POSSIBLY_COMPLETE_SILENCE_LENGTH_MILLIS, 800)
            putExtra(RecognizerIntent.EXTRA_SPEECH_INPUT_MINIMUM_LENGTH_MILLIS, 800)
        }
        recognizer?.startListening(intent)
    }

    fun destroy() { recognizer?.destroy(); recognizer = null }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/Tts.kt =====
package com.xiaoling.core

import android.content.Context
import android.speech.tts.TextToSpeech
import android.speech.tts.UtteranceProgressListener
import java.util.Locale

/** ?? TTS ??:?? init?????????? id ?????(??? flush ???????) */
class Tts(ctx: Context, private val onDone: (String?) -> Unit) {
```

```
    private var ready = false
    private var tts: TextToSpeech? = null
    private var seq = 0

    init {
        tts = TextToSpeech(ctx.applicationContext) { status ->
            if (status == TextToSpeech.SUCCESS) {
                val r = tts?.setLanguage(Locale.CHINA) ?: TextToSpeech.LANG_NOT_SUPPORTED
                ready = r != TextToSpeech.LANG_MISSING_DATA && r != TextToSpeech.LANG_NOT_SUPPORTED
                val cb = onDone   // ??,??? UtteranceProgressListener.onDone ??????
                tts?.setOnUtteranceProgressListener(object : UtteranceProgressListener() {
                    override fun onStart(id: String?) {}
                    override fun onDone(id: String?) { cb(id) }
                    @Deprecated("deprecated") override fun onError(id: String?) { cb(id) }
                })
            }
        }
    }

    val isReady: Boolean get() = ready

    /** ??;???? utteranceId?????/???????? onDone,??????? */
    fun speak(s: String): String {
        val id = (++seq).toString()
        if (ready && s.isNotBlank()) {
            val r = tts?.speak(s, TextToSpeech.QUEUE_FLUSH, null, id) ?: TextToSpeech.ERROR
            if (r == TextToSpeech.ERROR) onDone(id)
        } else {
            onDone(id)
        }
        return id
    }

    fun stop() { tts?.stop() }

    fun shutdown() { tts?.stop(); tts?.shutdown(); tts = null }
}

// ===== android-compose/app/src/main/java/com/xiaoling/core/WakeConfig.kt =====
package com.xiaoling.core

import android.content.Context

/**
 * ?????????? ? ???????(??);?? Picovoice AccessKey + ????/???
 * ?????????(?????????????)?
 *
 * ????(??):
 *  1. ? console.picovoice.ai ??,?? AccessKey;
 *  2. ???????????????,?? .ppn(? assets/wake/xiaoling_zh.ppn);
```

```
????????? V1.0   ? 27 ?
--------------------------------------------------------------------------------------------
 *  3. ?????? porcupine_params_zh.pv(? assets/wake/porcupine_params_zh.pv);
 *  4. ? AccessKey ???? ACCESS_KEY(???? SharedPreferences ??)?
 */
object WakeConfig {
    // ? ??? Picovoice AccessKey;?????????
    const val ACCESS_KEY = ""

    const val KEYWORD_ASSET = "wake/xiaoling_zh.ppn"
    const val MODEL_ASSET = "wake/porcupine_params_zh.pv"

    fun accessKey(ctx: Context): String {
        val ov = ctx.getSharedPreferences("xiaoling", Context.MODE_PRIVATE)
            .getString("pv_access_key", null)
        return if (!ov.isNullOrBlank()) ov else ACCESS_KEY
    }

    /** ?????(key + ?? asset ??)????? */
    fun available(ctx: Context): Boolean {
        if (accessKey(ctx).isBlank()) return false
        return assetExists(ctx, KEYWORD_ASSET) && assetExists(ctx, MODEL_ASSET)
    }

    private fun assetExists(ctx: Context, path: String): Boolean =
        try { ctx.assets.open(path).close(); true } catch (e: Exception) { false }
}

// ===== android-compose/app/src/main/java/com/xiaoling/service/FraudCallScreeningService.kt =====
package com.xiaoling.service

import android.telecom.Call
import android.telecom.CallScreeningService
import com.xiaoling.core.AlarmBus
import com.xiaoling.core.FraudStore
import com.xiaoling.core.NumberReputation
import com.xiaoling.core.Notifier

/**
 * ?????(Android 10+ ????????/????????)?
 * ?????:block=??,warn=????? ? ?? + ?? + ? AlarmBus ????????
 * ??:??? App ??????????,?????????
 */
class FraudCallScreeningService : CallScreeningService() {

    override fun onScreenCall(callDetails: Call.Details) {
        val number = callDetails.handle?.schemeSpecificPart ?: ""
        val (level, reason) = NumberReputation.assess(number)
        val builder = CallResponse.Builder()

        if (level == "block" || level == "warn") {
            if (level == "block") {
```

```
                builder.setDisallowCall(true).setRejectCall(true)
                    .setSkipCallLog(false).setSkipNotification(false)
            }
            val app = applicationContext
            val why = if (level == "block") "?????:$reason" else "$reason,???????????"
            FraudStore.inc(app)
            FraudStore.setPending(app, "????????:$reason")
            AlarmBus.post("????????:$reason")
            Notifier.warn(app, "? ??????", "$number ? $why", ("call" + number).hashCode())
        }
        respondToCall(callDetails, builder.build())
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/service/SmsFraudReceiver.kt =====
package com.xiaoling.service

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.provider.Telephony
import com.xiaoling.core.AlarmBus
import com.xiaoling.core.FraudStore
import com.xiaoling.core.FraudText
import com.xiaoling.core.Notifier

/**
 * ???????,?????????? ? ?? + ?????? + ? AlarmBus ????????
 * ? RECEIVE_SMS ??;??????????? SMS_RECEIVED(??,???)?
 */
class SmsFraudReceiver : BroadcastReceiver() {
    override fun onReceive(ctx: Context, intent: Intent) {
        if (intent.action != Telephony.Sms.Intents.SMS_RECEIVED_ACTION) return
        val msgs = Telephony.Sms.Intents.getMessagesFromIntent(intent) ?: return
        val text = msgs.joinToString("") { it.messageBody ?: "" }
        if (text.isBlank()) return
        val sender = msgs.firstOrNull()?.originatingAddress ?: "????"

        val (high, reason) = FraudText.assess(text)
        if (!high) return

        val app = ctx.applicationContext
        FraudStore.inc(app)
        FraudStore.setPending(app, "????????:$reason")
        AlarmBus.post("????????:$reason")
        Notifier.warn(
            app,
            "? ??????",
            "$sender:$reason?????????????????????",
            ("sms" + sender).hashCode()
```

```
        )
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/service/WakeService.kt =====
package com.xiaoling.service

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.os.Handler
import android.os.IBinder
import android.os.Looper
import android.speech.RecognitionListener
import android.speech.RecognizerIntent
import android.speech.SpeechRecognizer
import androidx.core.app.NotificationCompat
import com.xiaoling.MainActivity

/** App ?????(???? App ???,???????????,????) */
object AppForeground { @Volatile var active = false }

/**
 * ??????:?? App ?????,????????????,???? App ?????????
 * ??????????(???? Google ????;????????? Picovoice/Vosk ??)?
 * App ?????????(???)?
 */
class WakeService : Service() {

    private val main = Handler(Looper.getMainLooper())
    private var recognizer: SpeechRecognizer? = null
    @Volatile private var running = false
    private var porcupine: com.xiaoling.core.PorcupineWakeEngine? = null
    @Volatile private var offlineOn = false

    override fun onCreate() {
        super.onCreate()
        try {
            startForeground(NOTIF_ID, buildNotification())
        } catch (e: Exception) {
            stopSelf(); return   // ????????:????,??
        }
        running = true
        // ??????(Picovoice):???????????;???????????
        porcupine = com.xiaoling.core.PorcupineWakeEngine(this)
```

```
?????????? V1.0   ? 30 ?
------------------------------------------------------------------------------------------
        offlineOn = porcupine?.start { if (!AppForeground.active) wakeUp() } == true
        if (!offlineOn) loop()
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int = START_STICKY

    /** ??????:App ???/??????????;?????,????????? */
    private fun loop() {
        if (!running) return
        if (AppForeground.active || !SpeechRecognizer.isRecognitionAvailable(this)) {
            main.postDelayed({ loop() }, 1500); return
        }
        recognizer?.destroy()
        recognizer = SpeechRecognizer.createSpeechRecognizer(this).also { r ->
            r.setRecognitionListener(object : RecognitionListener {
                override fun onResults(results: Bundle) {
                    val hit = results.getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION)
                        ?.any { it.replace(" ", "").contains("??") } == true
                    if (hit) wakeUp() else again(500)
                }
                override fun onError(error: Int) = again(700)
                override fun onReadyForSpeech(params: Bundle?) {}
                override fun onBeginningOfSpeech() {}
                override fun onRmsChanged(rmsdB: Float) {}
                override fun onBufferReceived(buffer: ByteArray?) {}
                override fun onEndOfSpeech() {}
                override fun onPartialResults(partialResults: Bundle?) {}
                override fun onEvent(eventType: Int, params: Bundle?) {}
            })
        }
        val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
            putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL, RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
            putExtra(RecognizerIntent.EXTRA_LANGUAGE, "zh-CN")
            putExtra(RecognizerIntent.EXTRA_MAX_RESULTS, 1)
        }
        try { recognizer?.startListening(intent) } catch (e: Exception) { again(1000) }
    }

    private fun again(delay: Long) { main.postDelayed({ loop() }, delay) }

    /** ??:? App ?????????? */
    private fun wakeUp() {
        val i = Intent(this, MainActivity::class.java).apply {
            addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_REORDER_TO_FRONT)
            putExtra(EXTRA_WAKE, true)
        }
        try { startActivity(i) } catch (e: Exception) {}
        again(2500)   // ???????,?? App ?????
    }
```

```
    // ??:???????/??(????/??????);?????
    val ctx = androidx.compose.ui.platform.LocalContext.current
    androidx.compose.runtime.DisposableEffect(Unit) {
        val win = (ctx as? android.app.Activity)?.window
        win?.setFlags(android.view.WindowManager.LayoutParams.FLAG_SECURE,
            android.view.WindowManager.LayoutParams.FLAG_SECURE)
        onDispose { win?.clearFlags(android.view.WindowManager.LayoutParams.FLAG_SECURE) }
    }

    Box(
        Modifier.fillMaxSize().background(Brush.verticalGradient(listOf(BgTop, BgMid, BgBottom)))
    ) {
        Text("? ??", fontSize = 16.sp, color = AccentBlue,
            modifier = Modifier.align(Alignment.TopStart).padding(20.dp)
                .clickable { vm.showScreen(Screen.Settings) })

        Column(
            Modifier.fillMaxSize().padding(28.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Text("??", fontSize = 56.sp)
            Spacer(Modifier.height(10.dp))
            Text("????", fontSize = 26.sp, fontWeight = FontWeight.Bold, color = InkColor)
            Text("??????????????", fontSize = 13.sp, color = DimColor,
                modifier = Modifier.padding(top = 4.dp, bottom = 28.dp))

            OutlinedTextField(
                value = phone, onValueChange = { phone = it.filter { c -> c.isDigit() }.take(11) },
                label = { Text("???") }, singleLine = true,
                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Phone),
                modifier = Modifier.fillMaxWidth()
            )
            Spacer(Modifier.height(12.dp))
            OutlinedTextField(
                value = code, onValueChange = { code = it.filter { c -> c.isDigit() }.take(6) },
                label = { Text("???(??:1234)") }, singleLine = true,
                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
                modifier = Modifier.fillMaxWidth()
            )
            Spacer(Modifier.height(8.dp))
            OutlinedButton(
                onClick = { if (phone.length >= 6) { vm.sendCode(phone); sent = true } },
                modifier = Modifier.fillMaxWidth().height(46.dp), shape = RoundedCornerShape(14.dp)
            ) { Text(if (sent) "??????(?? 1234)" else "?????") }

            Spacer(Modifier.height(18.dp))
            Button(
                onClick = { if (phone.length >= 6 && code.isNotBlank()) vm.login(phone, code) },
                modifier = Modifier.fillMaxWidth().height(54.dp), shape = RoundedCornerShape(16.dp)
```

```
????????? V1.0   ? 42 ?
--------------------------------------------------------------------------------------------
            ) { Text("?? / ??", fontSize = 18.sp) }

            Spacer(Modifier.height(14.dp))
            Text("? ? ?", fontSize = 12.sp, color = DimColor)
            Spacer(Modifier.height(14.dp))
            Button(
                onClick = { vm.wxLogin() },
                modifier = Modifier.fillMaxWidth().height(52.dp), shape = RoundedCornerShape(16.dp),
                colors = androidx.compose.material3.ButtonDefaults.buttonColors(
                    containerColor = androidx.compose.ui.graphics.Color(0xFF07C160), contentColor = and
                )
            ) { Text("??????", fontSize = 18.sp) }

            if (ui.caption.contains("???") || ui.caption.contains("??")) {
                Spacer(Modifier.height(12.dp))
                Text(ui.caption, fontSize = 13.sp, color = DimColor, textAlign = TextAlign.Center)
            }
            Spacer(Modifier.height(20.dp))
            Text("????????????????????", fontSize = 11.sp, color = DimColor)
        }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/ui/SettingsScreen.kt =====
package com.xiaoling.ui

import android.app.role.RoleManager
import android.os.Build
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.Card
```

```
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Switch
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import com.xiaoling.R
import com.xiaoling.core.AppState
import com.xiaoling.core.Screen
import com.xiaoling.ui.theme.AccentBlue
import com.xiaoling.ui.theme.AccentGlow
import com.xiaoling.ui.theme.BgBottom
import com.xiaoling.ui.theme.BgMid
import com.xiaoling.ui.theme.BgTop
import com.xiaoling.ui.theme.DimColor
import com.xiaoling.ui.theme.InkColor

private val TABS = listOf("????", "????", "????")

@Composable
fun SettingsScreen(vm: AppState) {
    val ui by vm.state.collectAsStateWithLifecycle()
    val ctx = LocalContext.current
    var tab by remember { mutableIntStateOf(0) }
    var payPlan by remember { mutableStateOf<String?>(null) }
    var dialog by remember { mutableStateOf<Pair<String, String>?>(null) }

    val roleLauncher = rememberLauncherForActivityResult(
        ActivityResultContracts.StartActivityForResult()
    ) { }
```

```
????????? V1.0   ? 44 ?
-------------------------------------------------------------------------------------
    fun requestScreenRole() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
            val rm = ctx.getSystemService(RoleManager::class.java)
            if (rm != null && rm.isRoleAvailable(RoleManager.ROLE_CALL_SCREENING)) {
                roleLauncher.launch(rm.createRequestRoleIntent(RoleManager.ROLE_CALL_SCREENING))
            }
        }
    }

    Box(
        Modifier.fillMaxSize().background(Brush.verticalGradient(listOf(BgTop, BgMid, BgBottom)))
    ) {
        Column(Modifier.fillMaxSize()) {
            // ??
            Row(
                Modifier.fillMaxWidth().padding(start = 8.dp, end = 18.dp, top = 14.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text("?", fontSize = 32.sp, color = AccentBlue,
                    modifier = Modifier.clickable { vm.showScreen(Screen.Home) }.padding(horizontal = 1
                Text("??", fontSize = 22.sp, fontWeight = FontWeight.Bold, color = InkColor)
            }
            // ??
            TabBar(tab) { tab = it }

            Column(
                Modifier.fillMaxSize().verticalScroll(rememberScrollState()).padding(horizontal = 16.dp
            ) {
                Spacer(Modifier.height(6.dp))
                when (tab) {
                    0 -> UserTab(ui, showText = { title, body -> dialog = title to body },
                        onLogin = { vm.showScreen(Screen.Login) }, onLogout = { vm.logout() })
                    1 -> MemberTab(ui) { payPlan = it }
                    2 -> CareTab(ui, onSync = { vm.syncFamily() }, onRole = { requestScreenRole() },
                        onLive2d = { vm.setLive2d(it) }, onUpgrade = { tab = 1 })
                }
                Spacer(Modifier.height(28.dp))
                Text("?? ? v1.0", fontSize = 12.sp, color = DimColor,
                    modifier = Modifier.fillMaxWidth(), textAlign = TextAlign.Center)
                Spacer(Modifier.height(24.dp))
            }
        }

        // ????
        payPlan?.let { p ->
            AlertDialog(
                onDismissRequest = { payPlan = null },
                title = { Text("??????") },
                text = { Text("?? " + memberLabel(p) + "(" + (if (p == "basic") "?29.9/?" else "?299/?"
                confirmButton = { TextButton(onClick = { vm.buyPlan(p, "????"); payPlan = null }) { Tex
```

```
????????? V1.0   ? 45 ?
------------------------------------------------------------------------------------------
                dismissButton = { TextButton(onClick = { vm.buyPlan(p, "???"); payPlan = null }) { Text
            )
        }
        // ????(????/???)
        dialog?.let { (title, body) ->
            AlertDialog(
                onDismissRequest = { dialog = null },
                title = { Text(title) },
                text = { Text(body, fontSize = 15.sp, lineHeight = 23.sp) },
                confirmButton = { TextButton(onClick = { dialog = null }) { Text("???") } }
            )
        }
    }
}

@Composable
private fun TabBar(selected: Int, onSelect: (Int) -> Unit) {
    Row(
        Modifier.fillMaxWidth().padding(16.dp)
            .clip(RoundedCornerShape(16.dp)).background(Color.White.copy(alpha = 0.6f)).padding(4.dp)
    ) {
        TABS.forEachIndexed { i, t ->
            val on = i == selected
            Box(
                Modifier.weight(1f).clip(RoundedCornerShape(13.dp))
                    .background(if (on) AccentBlue else Color.Transparent)
                    .clickable { onSelect(i) }.padding(vertical = 11.dp),
                contentAlignment = Alignment.Center
            ) {
                Text(t, fontSize = 15.sp, fontWeight = if (on) FontWeight.Bold else FontWeight.Normal,
                    color = if (on) Color.White else DimColor)
            }
        }
    }
}

/* ---------------- ???:???? ---------------- */
@Composable
private fun UserTab(
    ui: com.xiaoling.core.UiState,
    showText: (String, String) -> Unit,
    onLogin: () -> Unit,
    onLogout: () -> Unit
) {
    GlassCard {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Image(
                painter = painterResource(R.drawable.avatar),
                contentDescription = null,
                contentScale = ContentScale.Crop,
```

```
                alignment = Alignment.TopCenter,
                modifier = Modifier.size(60.dp).clip(CircleShape).background(Color.White)
            )
            Spacer(Modifier.size(14.dp))
            Column(Modifier.weight(1f)) {
                Text(if (ui.loggedIn) "???" else "???", fontSize = 20.sp, fontWeight = FontWeight.Bold)
                Text(
                    (if (ui.loggedIn) maskPhone(ui.phone) else "?????????????") + " ? " + memberLabel(u
                    fontSize = 13.sp, color = DimColor
                )
            }
            if (ui.loggedIn) {
                Text("??", fontSize = 14.sp, color = AccentBlue,
                    modifier = Modifier.clickable { onLogout() }.padding(6.dp))
            } else {
                Text("??", fontSize = 14.sp, fontWeight = FontWeight.Bold, color = AccentBlue,
                    modifier = Modifier.clickable { onLogin() }.padding(6.dp))
            }
        }
    }
    GlassCard {
        RowItem("?????") { if (ui.loggedIn) showText("?????", "? ???:" + maskPhone(ui.phone) + "\n? ???
        Divider()
        RowItem("??????") { showText("??????", "? ??/???????????,?????\n? ???????????\n? ?????????????
        Divider()
        RowItem("????") { showText("????", "??????!?????? feedback@xiaoling.ai,?????????????") }
        Divider()
        RowItem("??????") { showText("??????", "??????????????????????????;???????????;?????????????????
    }
}

private fun maskPhone(p: String): String =
    if (p.length == 11) p.substring(0, 3) + "****" + p.substring(7) else p


/* ---------------- ???:???? ---------------- */
@Composable
private fun MemberTab(ui: com.xiaoling.core.UiState, onBuy: (String) -> Unit) {
    GlassCard {
        Row(Modifier.fillMaxWidth(), Arrangement.SpaceBetween, Alignment.CenterVertically) {
            Text("??????", fontSize = 20.sp, fontWeight = FontWeight.Bold)
            Text("??:" + memberLabel(ui.membership), fontSize = 14.sp, fontWeight = FontWeight.Bold,
                color = if (ui.membership.isEmpty()) DimColor else AccentBlue)
        }
        Spacer(Modifier.height(12.dp))
        PlanCard("????", "?29.9 / ?", listOf("??????", "??????", "?? + ????"),
            plan = "basic", current = ui.membership == "basic", onBuy = onBuy)
        Spacer(Modifier.height(12.dp))
        PlanCard("????", "?299 / ?",
            listOf("????????", "?? / ?????", "3D ?????", "???? ? ????", "??????"),
```

```
????????? V1.0   ? 47 ?
------------------------------------------------------------------------------------------
            plan = "premium", highlight = true, current = ui.membership == "premium", onBuy = onBuy)
    }
}

/* ---------------- ???:???? ---------------- */
@Composable
private fun CareTab(
    ui: com.xiaoling.core.UiState, onSync: () -> Unit, onRole: () -> Unit,
    onLive2d: (Boolean) -> Unit, onUpgrade: () -> Unit
) {
    val premium = ui.membership == "premium"
    GlassCard {
        Row(Modifier.fillMaxWidth(), Arrangement.SpaceBetween, Alignment.CenterVertically) {
            Text("????", fontSize = 20.sp, fontWeight = FontWeight.Bold)
            if (!premium) LockTag(onUpgrade)
        }
        Spacer(Modifier.height(8.dp))
        StatRow("?????????", "${ui.fraudBlocked} ?")   // ????:??
        StatRow("??????", ui.sosLabel)
        StatRow("?????", if (!premium) "????" else if (ui.familySynced) "??? ?" else "???")
        Spacer(Modifier.height(12.dp))
        Button(onClick = onSync, modifier = Modifier.fillMaxWidth().height(48.dp),
            shape = RoundedCornerShape(14.dp)) { Text(if (premium) "?????" else "?????(????)", fontSize
        Spacer(Modifier.height(8.dp))
        OutlinedButton(onClick = onRole, modifier = Modifier.fillMaxWidth().height(48.dp),
            shape = RoundedCornerShape(14.dp)) { Text("????????") }
    }
    GlassCard {
        Row(Modifier.fillMaxWidth(), Arrangement.SpaceBetween, Alignment.CenterVertically) {
            Column(Modifier.weight(1f)) {
                Text("3D ?????", fontSize = 18.sp, fontWeight = FontWeight.Bold)
                Text(if (premium) "?? VRM ?????,?? NATIVE.md" else "?????? ? ?????",
                    fontSize = 12.sp, color = DimColor, modifier = Modifier.padding(top = 2.dp))
            }
            if (premium) Switch(checked = ui.live2d, onCheckedChange = onLive2d)
            else LockTag(onUpgrade)
        }
    }
    GlassCard {
        Row(Modifier.fillMaxWidth(), Arrangement.SpaceBetween, Alignment.CenterVertically) {
            Column(Modifier.weight(1f)) {
                Text("?? / ?????", fontSize = 18.sp, fontWeight = FontWeight.Bold)
                Text(if (premium) "???,?????????" else "?????? ? ???????????",
                    fontSize = 12.sp, color = DimColor, modifier = Modifier.padding(top = 2.dp))
            }
            if (!premium) LockTag(onUpgrade) else Text("?", fontSize = 20.sp, color = DimColor)
        }
    }
}
```

```
????????? V1.0   ? 48 ?
-------------------------------------------------------------------------------------------
@Composable
private fun LockTag(onUpgrade: () -> Unit) {
    Text("?? ???", fontSize = 13.sp, fontWeight = FontWeight.Bold, color = Color(0xFFB8860B),
        modifier = Modifier.clip(RoundedCornerShape(999.dp)).background(Color(0xFFFFF0CC))
            .clickable { onUpgrade() }.padding(horizontal = 12.dp, vertical = 6.dp))
}


/* ---------------- ???? ---------------- */
@Composable
private fun GlassCard(content: @Composable androidx.compose.foundation.layout.ColumnScope.() -> Unit) {
    Card(
        modifier = Modifier.fillMaxWidth().padding(vertical = 7.dp),
        shape = RoundedCornerShape(22.dp),
        colors = CardDefaults.cardColors(containerColor = Color.White.copy(alpha = 0.82f)),
        elevation = CardDefaults.cardElevation(defaultElevation = 1.dp)
    ) { Column(Modifier.padding(18.dp), content = content) }
}

@Composable
private fun RowItem(title: String, onClick: () -> Unit) {
    Row(
        Modifier.fillMaxWidth().clickable { onClick() }.padding(vertical = 13.dp),
        horizontalArrangement = Arrangement.SpaceBetween, verticalAlignment = Alignment.CenterVerticall
    ) {
        Text(title, fontSize = 16.sp, color = InkColor)
        Text("?", fontSize = 20.sp, color = DimColor)
    }
}

@Composable
private fun Divider() {
    Box(Modifier.fillMaxWidth().height(1.dp).background(Color(0x11000000)))
}

@Composable
private fun StatRow(title: String, value: String) {
    Row(Modifier.fillMaxWidth().padding(vertical = 6.dp), Arrangement.SpaceBetween) {
        Text(title, fontSize = 16.sp, color = MaterialTheme.colorScheme.onSurface)
        Text(value, fontSize = 16.sp, fontWeight = FontWeight.Bold, color = AccentBlue)
    }
}

@Composable
private fun PlanCard(
    name: String, price: String, benefits: List<String>, plan: String,
    highlight: Boolean = false, current: Boolean = false, onBuy: (String) -> Unit
) {
    Column(
        Modifier.fillMaxWidth().clip(RoundedCornerShape(18.dp))
```

```
?????????? V1.0   ? 49 ?
------------------------------------------------------------------------------------------
            .background(
                if (highlight) Brush.verticalGradient(listOf(Color(0xFFFFF3DC), Color(0xFFFFF9EF)))
                else Brush.verticalGradient(listOf(Color(0xFFEDF1FF), Color(0xFFF7F9FF)))
            ).padding(16.dp)
    ) {
        Row(Modifier.fillMaxWidth(), Arrangement.SpaceBetween, Alignment.CenterVertically) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Text(name, fontSize = 18.sp, fontWeight = FontWeight.Bold)
                if (highlight) {
                    Spacer(Modifier.size(6.dp))
                    Text("??", fontSize = 11.sp, color = Color(0xFF8A5A00),
                        modifier = Modifier.clip(RoundedCornerShape(6.dp)).background(Color(0xFFFFE0A3)
                }
            }
            Text(price, fontSize = 18.sp, fontWeight = FontWeight.Bold,
                color = if (highlight) Color(0xFFB8860B) else AccentBlue)
        }
        Spacer(Modifier.height(6.dp))
        benefits.forEach { Text("? $it", fontSize = 14.sp, color = DimColor, modifier = Modifier.paddin
        Spacer(Modifier.height(10.dp))
        Button(
            onClick = { if (!current) onBuy(plan) }, enabled = !current,
            modifier = Modifier.fillMaxWidth().height(46.dp), shape = RoundedCornerShape(13.dp)
        ) { Text(if (current) "???" else "????", fontSize = 16.sp) }
    }
}

private fun memberLabel(tier: String): String = when (tier) {
    "basic" -> "????"; "premium" -> "????"; else -> "???"
}

// ===== android-compose/app/src/main/java/com/xiaoling/ui/XiaolingApp.kt =====
package com.xiaoling.ui

import androidx.activity.compose.BackHandler
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import com.xiaoling.core.AppState
import com.xiaoling.core.Screen
import com.xiaoling.ui.theme.XiaolingTheme

@Composable
fun XiaolingApp(vm: AppState) {
    XiaolingTheme {
        Surface(
```

```
????????? V1.0   ? 50 ?
--------------------------------------------------------------------------------------------
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background
        ) {
            val ui by vm.state.collectAsStateWithLifecycle()
            // ?????:??/????????,????? App
            BackHandler(enabled = ui.screen == Screen.Settings) { vm.showScreen(Screen.Home) }
            BackHandler(enabled = ui.screen == Screen.Login) { vm.showScreen(Screen.Settings) }
            when (ui.screen) {
                Screen.Home -> HomeScreen(vm)
                Screen.Settings -> SettingsScreen(vm)
                Screen.Login -> LoginScreen(vm)
            }
        }
    }
}

// ===== android-compose/app/src/main/java/com/xiaoling/ui/theme/Theme.kt =====
package com.xiaoling.ui.theme

import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Typography
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

// ??? ? ??? ? ???????:????,?????
val BgTop = Color(0xFFFFFFFF)
val BgMid = Color(0xFFF4F8FF)
val BgBottom = Color(0xFFEAF2FF)
val AccentBlue = Color(0xFF2F6BFF)
val AccentGlow = Color(0xFF4EA8FF)
val InkColor = Color(0xFF16233B)
val DimColor = Color(0xFF8A93A6)
val AlarmRed = Color(0xFFE5484D)
val CardBg = Color(0xFFFFFFFF)

private val Scheme = lightColorScheme(
    primary = AccentBlue,
    onPrimary = Color.White,
    secondary = AccentGlow,
    background = BgBottom,
    onBackground = InkColor,
    surface = CardBg,
    onSurface = InkColor,
    onSurfaceVariant = DimColor,
    error = AlarmRed
)
```

```
????????? V1.0   ? 51 ?
-------------------------------------------------------------------------------------------
```

```
private val AppType = Typography(
    titleLarge = TextStyle(fontSize = 27.sp, fontWeight = FontWeight.Bold, letterSpacing = 0.2.sp),
    titleMedium = TextStyle(fontSize = 19.sp, fontWeight = FontWeight.SemiBold),
    bodyLarge = TextStyle(fontSize = 17.sp, letterSpacing = 0.15.sp),
    bodyMedium = TextStyle(fontSize = 14.sp, color = DimColor)
)

@Composable
fun XiaolingTheme(content: @Composable () -> Unit) {
    MaterialTheme(colorScheme = Scheme, typography = AppType, content = content)
}
```

```
// ===== server/firewall.py =====
"""
?? ? ?????(?????,? Starlette ???)
??:
  - ??(???,???? IP,?? + ??????)
  - ???????(??? body ??)
  - ?????(HSTS / nosniff / ??????)
  - ??/????????(??)
????:?????? Nginx/Cloudflare/? WAF;??????????
"""
from __future__ import annotations
import time
from collections import defaultdict, deque

from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse, Response

# ---- ?? ----
MAX_BODY_BYTES = 64 * 1024          # ?????? 64KB(??/????)
GLOBAL_RATE = (120, 60)             # ? IP:? 60s ?? 120 ?
SENSITIVE_RATE = (10, 60)           # ????(??/??/??):? 60s ?? 10 ?
SENSITIVE_PREFIXES = ("/auth/", "/pay/")
# SSE ???????????????
STREAM_PREFIXES = ("/push/subscribe",)


class _Bucket:
    """???????"""
    def __init__(self):
        self.hits: dict[str, deque] = defaultdict(deque)

    def allow(self, key: str, limit: int, window: int, now: float) -> bool:
        q = self.hits[key]
        cutoff = now - window
        while q and q[0] < cutoff:
            q.popleft()
```

```
????????? V1.0   ? 52 ?
--------------------------------------------------------------------------------------------
        if len(q) >= limit:
            return False
        q.append(now)
        return True


_global = _Bucket()
_sensitive = _Bucket()
# ???????:???????
_last_gc = [0.0]


def _client_ip(request: Request) -> str:
    # ?????,?? X-Forwarded-For ???(??????????)
    xff = request.headers.get("x-forwarded-for")
    if xff:
        return xff.split(",")[0].strip()
    return request.client.host if request.client else "unknown"


class Firewall(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        now = time.time()
        ip = _client_ip(request)
        path = request.url.path

        # 1) ?????(? Content-Length ??,?????)
        if not path.startswith(STREAM_PREFIXES):
            cl = request.headers.get("content-length")
            if cl and cl.isdigit() and int(cl) > MAX_BODY_BYTES:
                return JSONResponse({"ok": False, "error": "request too large"}, status_code=413)

        # 2) ??:?? + ??????
        g_limit, g_win = GLOBAL_RATE
        if not _global.allow(ip, g_limit, g_win, now):
            return _rate_limited(g_win)
        if path.startswith(SENSITIVE_PREFIXES):
            s_limit, s_win = SENSITIVE_RATE
            if not _sensitive.allow(ip, s_limit, s_win, now):
                return _rate_limited(s_win)

        # 3) ??? GC,?????????
        if now - _last_gc[0] > 300:
            _last_gc[0] = now
            _gc(_global, now, g_win)
            _gc(_sensitive, now, SENSITIVE_RATE[1])

        resp: Response = await call_next(request)
        _harden(resp)
        return resp
```

```
????????? V1.0   ? 53 ?
--------------------------------------------------------------------------------


def _rate_limited(window: int) -> JSONResponse:
    r = JSONResponse({"ok": False, "error": "too many requests"}, status_code=429)
    r.headers["Retry-After"] = str(window)
    _harden(r)
    return r


def _harden(resp: Response) -> None:
    resp.headers.setdefault("X-Content-Type-Options", "nosniff")
    resp.headers.setdefault("X-Frame-Options", "DENY")
    resp.headers.setdefault("Referrer-Policy", "no-referrer")
    resp.headers.setdefault("Strict-Transport-Security", "max-age=31536000; includeSubDomains")
    resp.headers.setdefault("Cache-Control", "no-store")


def _gc(bucket: _Bucket, now: float, window: int) -> None:
    cutoff = now - window
    dead = [k for k, q in bucket.hits.items() if not q or q[-1] < cutoff]
    for k in dead:
        bucket.hits.pop(k, None)


def install(app) -> None:
    """? main.py ? install(app) ??????"""
    app.add_middleware(Firewall)

// ===== server/fraud.py =====
"""
?? ? ??????? v3
????(??):
  ???(???/????,???)
  ? L1a ?????????(??)
  ? L0 ??????
  ? L1b ????(????? + ????)
  ? ????(??/??/????/??)+ ????(????,???)
  ? ????
????:ConversationTracker ????(???????),???????
???????:??? / ?? / ?? / ??? / ???? / ??????
????? fraud_rules.json,??????
"""
from __future__ import annotations
import json
import os
from dataclasses import dataclass, field, asdict
from typing import Optional

_RULES_PATH = os.path.join(os.path.dirname(__file__), "fraud_rules.json")
```

```
????????? V1.0   ? 54 ?
--------------------------------------------------------------------------------------------

def _load_rules() -> dict:
    with open(_RULES_PATH, encoding="utf-8") as f:
        return json.load(f)


_RULES = _load_rules()


def reload_rules() -> str:
    """?????:????/?????????,??????"""
    global _RULES
    _RULES = _load_rules()
    return _RULES.get("version", "")


@dataclass
class FraudResult:
    risk: float = 0.0                # 0~1
    level: str = "safe"              # safe / medium / high
    category: str = ""               # ??????? label
    reason: str = ""                 # ?????????
    hits: list[str] = field(default_factory=list)
    amplifiers: list[str] = field(default_factory=list)  # ???????
    suggest_hangup: bool = False
    report: Optional[dict] = None     # ????????????

    def to_dict(self) -> dict:
        return asdict(self)


def normalize(text: str) -> str:
    """???:??/??/????? ? ???,????????"""
    t = text or ""
    nm = _RULES.get("normalize", {}).get("map", {})
    # ?????????,?????????
    for variant, std in nm.items():
        if variant in t:
            t = t.replace(variant, std)
    # ?????????(????"? ? ?"??),??????
    compact = t.replace(" ", "").replace("?", "")
    for variant, std in nm.items():
        cv = variant.replace(" ", "")
        if cv in compact:
            compact = compact.replace(cv, std)
    return compact


def analyze(text: str, caller: str = "", scene: str = "incoming_call") -> FraudResult:
    raw = text or ""
```

```
????????? V1.0   ? 55 ?
--------------------------------------------------------------------------------------------
    t = normalize(raw)
    cats = _RULES["categories"]
    th = _RULES["thresholds"]

    # ?? L1a ???:??????,????(??????????)??
    red = cats["redline"]
    red_hits = [w for w in red["words"] if w in t]
    if red_hits:
        amps = _amplifier_hits(t)
        return _finalize(0.96, red["label"], red_hits, amps,
                         f"?????{red_hits[0]}?,????????????", caller, raw, scene, th)

    # ?? L0 ?????? ??
    base = 0.15 if _suspicious_number(caller) else 0.0

    # ?? L1b ????:????? + ???? ??
    best_cat, best_score, all_hits = "", 0.0, []
    for key, c in cats.items():
        if key == "redline":
            continue
        hits = [w for w in c["words"] if w in t]
        if not hits:
            continue
        all_hits += hits
        score = c["weight"] + 0.1 * (len(hits) - 1)
        if score > best_score:
            best_score, best_cat = score, c["label"]

    risk = base + best_score

    # ?? ?? / ???? ??
    amp_hits = _amplifier_hits(t)
    risk += sum(_RULES["amplifiers"]["signals"][k]["add"] for k in amp_hits)
    supp = _suppressor_delta(t)
    risk -= supp

    risk = max(0.0, min(risk, 0.99))

    if not all_hits and risk < th["medium"]:
        return FraudResult(risk=round(risk, 2), level="safe", amplifiers=amp_hits)

    cat = best_cat or "????"
    reason = ("?????" + "?".join(all_hits[:3]) + "?" if all_hits else "??????") + \
             ("," + "?".join(_amp_labels(amp_hits)) if amp_hits else "") + ",?????"
    return _finalize(risk, cat, all_hits, amp_hits, reason, caller, raw, scene, th)


def _amplifier_hits(t: str) -> list[str]:
    out = []
    for name, sig in _RULES.get("amplifiers", {}).get("signals", {}).items():
```

```
        if any(w in t for w in sig["words"]):
            out.append(name)
    return out


def _amp_labels(keys: list[str]) -> list[str]:
    label = {"urgency": "????", "secrecy": "????", "money_action": "????/???", "authority": "????"}
    return [label.get(k, k) for k in keys]


def _suppressor_delta(t: str) -> float:
    total = 0.0
    for name, sig in _RULES.get("suppressors", {}).get("signals", {}).items():
        if any(w in t for w in sig["words"]):
            total += sig["sub"]
    return total


def _finalize(risk, category, hits, amps, reason, caller, text, scene, th) -> FraudResult:
    risk = max(0.0, min(risk, 0.99))
    level = "high" if risk >= th["high"] else "medium" if risk >= th["medium"] else "safe"
    report = None
    if level != "safe":
        report = {
            "scene": scene, "caller": caller, "category": category,
            "risk": round(risk, 2), "hits": hits, "amplifiers": amps, "snippet": text[:60],
        }
    return FraudResult(risk=round(risk, 2), level=level, category=category,
                       reason=reason, hits=hits, amplifiers=amps,
                       suggest_hangup=(level == "high"), report=report)


def _suspicious_number(caller: str) -> bool:
    if not caller:
        return False
    c = caller.replace("-", "").replace(" ", "")
    for p in _RULES["number_reputation"]["suspicious_prefixes"]:
        if c.startswith(p) and not c.startswith("+86"):
            return True
    digits = c.lstrip("+")
    return not (7 <= len(digits) <= 12)


class ConversationTracker:
    """
    ??????:????/?????,????????,???????,
    ??????????????????(??? decay ??),??????
    ??:???? add(text),???????? FraudResult?
    """
    def __init__(self, caller: str = "", scene: str = "incoming_call"):
```

```
        self.caller = caller
        self.scene = scene
        self.cum_risk = 0.0
        self.all_hits: list[str] = []
        self.all_amps: set[str] = set()
        self.category = ""
        self.turns = 0

    def add(self, text: str) -> FraudResult:
        conv = _RULES.get("conversation", {})
        decay = conv.get("decay_per_turn", 0.85)
        one = analyze(text, self.caller, self.scene)
        self.turns += 1
        # ?????,?????????(?????????????)
        self.cum_risk = max(one.risk, self.cum_risk * decay + one.risk * 0.5)
        self.cum_risk = min(self.cum_risk, 0.99)
        for h in one.hits:
            if h not in self.all_hits:
                self.all_hits.append(h)
        self.all_amps.update(one.amplifiers)
        if one.category and (not self.category or one.risk >= 0.5):
            self.category = one.category

        th = _RULES["thresholds"]
        reason = one.reason if one.risk >= self.cum_risk else \
            ("??????????:" + ("?".join(self.all_hits[:3]) if self.all_hits else "??????"))
        return _finalize(self.cum_risk, self.category or one.category or "????",
                         self.all_hits, list(self.all_amps), reason,
                         self.caller, text, self.scene, th)


# ?? ?????(skills.py ???)??
def analyze_fraud(caller: str, text: str) -> tuple[float, str]:
    r = analyze(text, caller)
    return r.risk, r.reason

// ===== server/fraud_session.py =====
"""
?? ? ???????
??? id(?? context.session_id,?? caller+scene)?? ConversationTracker,
?"????/????"???????? TTL ????,??????
?????/????????? Redis ??????
"""
from __future__ import annotations
import time
from fraud import ConversationTracker, FraudResult, analyze

_SESSIONS: dict[str, tuple[float, ConversationTracker]] = {}
_TTL = 180          # ?? 3 ????????
_MAX = 5000         # ???????(??????)
```

```
????????? V1.0   ? 58 ?
--------------------------------------------------------------------------------------------


def _session_id(context: dict, caller: str, scene: str) -> str:
    sid = (context or {}).get("session_id")
    if sid:
        return str(sid)
    return f"{scene}:{caller or 'unknown'}"


def _gc(now: float) -> None:
    dead = [k for k, (ts, _) in _SESSIONS.items() if now - ts > _TTL]
    for k in dead:
        _SESSIONS.pop(k, None)
    if len(_SESSIONS) > _MAX:
        # ???????
        for k, _ in sorted(_SESSIONS.items(), key=lambda kv: kv[1][0])[: len(_SESSIONS) - _MAX]:
            _SESSIONS.pop(k, None)


def analyze_session(text: str, caller: str, scene: str, context: dict | None = None) -> FraudResult:
    """
    ??:??????????????/????(?????)???? analyze?
    """
    if scene not in ("incoming_call", "sms", "incoming_sms"):
        return analyze(text, caller, scene)

    now = time.time()
    _gc(now)
    sid = _session_id(context or {}, caller, scene)
    entry = _SESSIONS.get(sid)
    if entry is None:
        tr = ConversationTracker(caller=caller, scene=scene)
        _SESSIONS[sid] = (now, tr)
    else:
        tr = entry[1]
        _SESSIONS[sid] = (now, tr)
    return tr.add(text)


def end_session(context: dict | None, caller: str = "", scene: str = "incoming_call") -> None:
    """??/????????,???????"""
    sid = _session_id(context or {}, caller, scene)
    _SESSIONS.pop(sid, None)

// ===== server/llm.py =====
"""
?? ? ?????(???? + ????)
????:
  - ??????????,? 60%+ ???????
  - ?? anthropic / ?? KEY ??????"??????",???????
```

```
????????? V1.0   ? 59 ?
------------------------------------------------------------------------------------------
  - ????? _call_anthropic ?? ??/??/?? ? Function-Calling ??,?????
"""
from __future__ import annotations
import os

from models import Utterance, Reply

# ?? ? ???????
_INTENT_TO_ACTION = {
    "call": "CALL", "navigate": "OPEN_URI", "play_music": "PLAY",
    "translate": "TRANSLATE", "chat": None, "unknown": None,
}

_TOOLS = [{
    "name": "dispatch",
    "description": "?????????????",
    "input_schema": {
        "type": "object",
        "properties": {
            "speech": {"type": "string",
                       "description": "???????????????,?????"},
            "intent": {"type": "string",
                       "enum": ["chat", "call", "navigate", "play_music",
                                "translate", "unknown"]},
            "slots": {"type": "object", "description": "? {'target':'???'}"},
        },
        "required": ["speech", "intent"],
    },
}]

_SYSTEM = ("???????????'??'???????????,"
           "????????????????????? dispatch?"
           "?????? intent=chat,????????")


def llm_reply(u: Utterance) -> Reply:
    try:
        data = _call_anthropic(u.text)
    except Exception:
        return _offline_fallback(u)      # ??/?KEY/??? ? ??
    intent = data.get("intent", "chat")
    slots = data.get("slots", {}) or {}
    action = None
    atype = _INTENT_TO_ACTION.get(intent)
    if atype:
        action = {"type": atype, **slots}
    return Reply(speech=data.get("speech", "????"), action=action, skill=f"llm:{intent}")


def _call_anthropic(text: str) -> dict:
```

```
????????? V1.0   ? 60 ?
-------------------------------------------------------------------------------------------
    import anthropic  # ??????? ? ???
    if not os.getenv("ANTHROPIC_API_KEY"):
        raise RuntimeError("no api key")
    client = anthropic.Anthropic()
    msg = client.messages.create(
        model="claude-opus-4-8",         # ??? qwen-max / doubao-pro
        max_tokens=512,
        system=_SYSTEM,
        tools=_TOOLS,
        tool_choice={"type": "tool", "name": "dispatch"},
        messages=[{"role": "user", "content": text}],
    )
    for block in msg.content:
        if getattr(block, "type", "") == "tool_use":
            return block.input
    return {"speech": "?????,????????", "intent": "unknown"}


def _offline_fallback(u: Utterance) -> Reply:
    """????????:???????????,????"""
    return Reply(
        speech="??????????????????????????,"
               "???????????????",
        skill="offline_fallback",
    )


# ---------- ??????(????????,???) ----------
_FRAUD_TOOL = [{
    "name": "judge_fraud",
    "description": "??????/???????????",
    "input_schema": {
        "type": "object",
        "properties": {
            "is_fraud": {"type": "boolean", "description": "?????"},
            "confidence": {"type": "number", "description": "0~1 ???"},
            "reason": {"type": "string", "description": "???????????"},
        },
        "required": ["is_fraud", "confidence"],
    },
}]

_FRAUD_SYSTEM = (
    "???????????????/???????????"
    "????:?????/??/??/?????????/???/?????"
    "?????????????????????????"
    "????(???????????????)? is_fraud=false???? judge_fraud?"
)
```

```
????????? V1.0   ? 61 ?
------------------------------------------------------------------------------------------
def judge_fraud(text: str, category: str = "") -> dict | None:
    """
    ????????? {is_fraud, confidence, reason} ? None(?????,???????)?
    ??????(???)???,????????
    """
    try:
        import anthropic
        if not os.getenv("ANTHROPIC_API_KEY"):
            return None
        client = anthropic.Anthropic()
        msg = client.messages.create(
            model="claude-opus-4-8",     # ??? qwen-max / doubao-pro
            max_tokens=256,
            system=_FRAUD_SYSTEM,
            tools=_FRAUD_TOOL,
            tool_choice={"type": "tool", "name": "judge_fraud"},
            messages=[{"role": "user", "content": f"[????:{category}] ??:{text}"}],
        )
        for block in msg.content:
            if getattr(block, "type", "") == "tool_use":
                return block.input
    except Exception:
        return None
    return None


# ---------- ????(????????) ----------
_LANG_CN = {"english": "??", "cantonese": "??", "mandarin": "???"}


def llm_translate(content: str, lang: str) -> str | None:
    """??????????/? KEY ?? None,?????????"""
    try:
        import anthropic
        if not os.getenv("ANTHROPIC_API_KEY"):
            return None
        target = _LANG_CN.get(lang, "??")
        client = anthropic.Anthropic()
        msg = client.messages.create(
            model="claude-opus-4-8",
            max_tokens=200,
            system=f"??????????????{target},?????,??????,?????",
            messages=[{"role": "user", "content": content}],
        )
        for block in msg.content:
            if getattr(block, "type", "") == "text":
                return block.text.strip()
    except Exception:
        return None
    return None
```

```
????????? V1.0   ? 62 ?
--------------------------------------------------------------------------------------------

// ===== server/main.py =====
"""
?? ? ??????
??:????(?)? ????(?),????? 0 ?????
??: cd server && uvicorn main:app --host 0.0.0.0 --port 8000
??: http://localhost:8000/docs
"""
from __future__ import annotations
from fastapi import FastAPI

from models import Utterance, Reply
import skills          # ?????????
from llm import llm_reply
import firewall        # ?????(??/????/???)

app = FastAPI(title="?? ? AI??????", version="0.1.0")
firewall.install(app)  # ????????


@app.get("/health")
def health():
    return {"ok": True, "skills": [name for name, _, _ in skills._REGISTRY]}


@app.post("/dialogue", response_model=Reply)
def dialogue(u: Utterance) -> Reply:
    """????:?????(???/??),???????"""
    r = skills.match(u)
    if r:
        return r
    return llm_reply(u)


# ? CLI / ?? ???????,????
def handle(text: str, context: dict | None = None, user_id: str = "guest") -> Reply:
    return dialogue(Utterance(text=text, context=context, user_id=user_id))


# ---------- ??(?????;????/??????? SDK + ??? + ??) ----------
from pydantic import BaseModel, Field
import time


class Order(BaseModel):
    plan: str = Field(default="basic", pattern="^(basic|premium)$")
    method: str = Field(default="", max_length=16)
    phone: str = Field(default="", max_length=32)
```

```
????????? V1.0   ? 63 ?
--------------------------------------------------------------------------------
# ?????(demo):{ phone: {membership, family_id, uid} }??????????
_users: dict[str, dict] = {}


class SendCode(BaseModel):
    phone: str = Field(..., min_length=6, max_length=20, pattern=r"^\+?\d{6,20}$")


class LoginReq(BaseModel):
    phone: str = Field(..., min_length=6, max_length=20, pattern=r"^\+?\d{6,20}$")
    code: str = Field(..., min_length=4, max_length=8, pattern=r"^\d{4,8}$")


@app.post("/auth/send_code")
def auth_send_code(s: SendCode):
    """??????????????(???/???)?demo:????? 1234?"""
    return {"ok": True, "hint": "?????? 1234(??????????)"}


@app.post("/auth/login")
def auth_login(r: LoginReq):
    """??? + ?????/???demo:??? 1234 ??????/????????"""
    if r.code != "1234":
        return {"ok": False, "msg": "?????(????? 1234)"}
    u = _users.setdefault(r.phone, {})
    u.setdefault("uid", "u" + str(abs(hash(r.phone)) % 1000000))
    u.setdefault("family_id", "fam-" + str(abs(hash(r.phone)) % 100000))
    u.setdefault("membership", "")
    return {"ok": True, "token": "demo-" + u["uid"], "uid": u["uid"],
            "family_id": u["family_id"], "membership": u["membership"]}


class WxLogin(BaseModel):
    code: str = Field(default="", max_length=128)


@app.post("/auth/wx_login")
def auth_wx_login(w: WxLogin):
    """???????????:??? code ??? code2session ? openid,?????demo:???????????"""
    phone = "wx-" + (w.code[-6:] if w.code else "demo")
    u = _users.setdefault(phone, {})
    u.setdefault("uid", "wx" + str(abs(hash(phone)) % 1000000))
    u.setdefault("family_id", "fam-" + str(abs(hash(phone)) % 100000))
    u.setdefault("membership", "")
    return {"ok": True, "token": "demo-" + u["uid"], "uid": u["uid"], "phone": phone,
            "family_id": u["family_id"], "membership": u["membership"]}


@app.post("/pay/create")
def pay_create(o: Order):
```

```
????????? V1.0   ? 64 ?
-------------------------------------------------------------------------------------------
    """??:?????????????/?????,?? prepay_id/orderInfo ?????????"""
    price = "29.9" if o.plan == "basic" else "299"
    if o.phone:                        # ??? ? ??????(??????)
        _users.setdefault(o.phone, {}).update(membership=o.plan)
    return {"ok": True, "orderId": f"XL{int(time.time())}", "plan": o.plan,
            "method": o.method, "amount": price}


@app.post("/pay/notify")
def pay_notify(body: dict):
    """????(???????????????,???????????)?"""
    return {"ok": True, "paid": True}


# ---------- ???????(????) ----------
# ???? + SSE ??:??????? ? ?????????
# ???????????(??/??/??/??/APNs)??????? App?;?? SSE ???App ????????
import asyncio
import json as _json
from collections import defaultdict
from fastapi import Request
from fastapi.responses import StreamingResponse

_subscribers: dict[str, list[asyncio.Queue]] = defaultdict(list)


class Event(BaseModel):
    family_id: str = Field(..., max_length=64)          # ??? id(???????)
    sender: str = Field(default="", max_length=64)      # ???? id(??????????????)
    type: str = Field(default="", max_length=32)        # fraud_call / fraud_sms / sos / meds / sync
    text: str = Field(default="", max_length=500)
    at: int = 0


@app.post("/push/emit")
async def push_emit(e: Event):
    """???????,???????????????"""
    payload = _json.dumps(e.model_dump(), ensure_ascii=False)
    for q in list(_subscribers.get(e.family_id, [])):
        try:
            q.put_nowait(payload)
        except Exception:
            pass
    return {"ok": True, "delivered": len(_subscribers.get(e.family_id, []))}


@app.get("/push/subscribe")
async def push_subscribe(family_id: str, request: Request):
    """???????????(SSE ???,????)?"""
    q: asyncio.Queue = asyncio.Queue()
```

```
    _subscribers[family_id].append(q)

    async def gen():
        try:
            yield "event: ready\ndata: {}\n\n"
            while True:
                if await request.is_disconnected():
                    break
                try:
                    data = await asyncio.wait_for(q.get(), timeout=15)
                    yield f"data: {data}\n\n"
                except asyncio.TimeoutError:
                    yield ": keep-alive\n\n"   # ??
        finally:
            try:
                _subscribers[family_id].remove(q)
            except ValueError:
                pass

    return StreamingResponse(gen(), media_type="text/event-stream")



// ===== server/models.py =====
"""
?? ? ???? ?? ????
?????????/?????
"""
from __future__ import annotations
from typing import Optional, Any
from pydantic import BaseModel, Field


class Utterance(BaseModel):
    """????????(ASR ??????)"""
    user_id: str = Field(default="guest", max_length=64)
    text: str = Field(..., max_length=2000, description="ASR ??????,???????")
    context: Optional[dict[str, Any]] = Field(
        default=None,
        description="????,? {'scene':'incoming_call','caller':'400xxxx'}",
    )


class Reply(BaseModel):
    """???????:??? speech,??? action"""
    speech: str = Field(..., description="?? TTS ????")
    action: Optional[dict[str, Any]] = Field(
        default=None, description="?????????,??? App / ??"
    )
    skill: str = Field(default="", description="??????,???????")
```

```
????????? V1.0   ? 66 ?
--------------------------------------------------------------------------------------------
    risk: float = Field(default=0.0, description="???(?????),0~1")

// ===== server/skills.py =====
"""
?? ? ?????
???? = ????,?? Utterance,????? Reply,????? None?
??????????? @skill ?? + ?? DeepLink,??????
???????(??/???),0 ?????;??????? llm.py?
"""
from __future__ import annotations
import re
from typing import Callable, Optional

from models import Utterance, Reply
from fraud import analyze as fraud_analyze
from fraud_session import analyze_session
from llm import judge_fraud, llm_translate
from translate import parse_translate, translate_phrase, LANGS

# (name, priority, fn):priority ?????,??/???????
_REGISTRY: list[tuple[str, int, Callable[[Utterance], Optional[Reply]]]] = []


def skill(name: str, priority: int = 100):
    def deco(fn: Callable[[Utterance], Optional[Reply]]):
        _REGISTRY.append((name, priority, fn))
        _REGISTRY.sort(key=lambda x: x[1])
        return fn
    return deco


def match(u: Utterance) -> Optional[Reply]:
    """????????,??????"""
    for name, _, fn in _REGISTRY:
        r = fn(u)
        if r:
            r.skill = r.skill or name
            return r
    return None


# ========================= ???? =========================

@skill("????", priority=1)
def sos(u: Utterance) -> Optional[Reply]:
    if not re.search(r"??|????|????|???|??|???|??120|??|?", u.text):
        return None
    return Reply(
        speech="??,???????120,????????,???????,?????",
        action={"type": "SOS", "call": "120", "notify_family": True, "send_location": True},
```

```
    )


@skill("?????", priority=2)
def anti_fraud(u: Utterance) -> Optional[Reply]:
    """??/????:context ? caller/scene/session_id,???????????"""
    ctx = u.context or {}
    scene = ctx.get("scene")
    if scene not in ("incoming_call", "sms", "incoming_sms"):
        return None
    # ??:?????/????????(?????????)
    r = analyze_session(u.text, ctx.get("caller", ""), scene, ctx)
    if r.level == "safe":
        return None
    # ??(?????)? ???????,???;??????????
    if r.level == "medium":
        verdict = judge_fraud(u.text, r.category)
        if verdict is not None:
            if not verdict.get("is_fraud", True) and verdict.get("confidence", 0) >= 0.6:
                return None   # ???????? ? ???
            if verdict.get("is_fraud") and verdict.get("confidence", 0) >= 0.85:
                r.level = "high"   # ???????? ? ??
    level_cn = "??" if r.level == "high" else "??"
    return Reply(
        speech=(f"??!????????{level_cn}(??{r.category}):{r.reason}?"
                f"??????????????????????????"
                f"????????,??????????"),
        action={"type": "FRAUD_WARN", "level": r.level, "category": r.category,
                "hangup_suggest": r.suggest_hangup, "report": r.report},
        risk=r.risk,
    )


@skill("????", priority=8)
def translate(u: Utterance) -> Optional[Reply]:
    """????:???/??/???????????,??????????"""
    parsed = parse_translate(u.text)
    if not parsed:
        return None
    content, lang = parsed
    lang_cn = LANGS.get(lang, "??")
    hit = translate_phrase(content, lang)
    if hit:
        return Reply(speech=f"{content} ?{lang_cn}?:{hit}",
                     action={"type": "SPEAK", "text": hit, "lang": lang}, skill="????")
    # ??? ? ?????(? KEY ?????)
    out = llm_translate(content, lang)
    if out:
        return Reply(speech=f"{content} ?{lang_cn}?:{out}",
                     action={"type": "SPEAK", "text": out, "lang": lang}, skill="????")
```

```
????????? V1.0   ? 68 ?
-------------------------------------------------------------------------------------------
    return Reply(speech=f"????????,????????", skill="????")


@skill("???", priority=10)
def call_phone(u: Utterance) -> Optional[Reply]:
    m = re.search(r"(?:?(?:?)?????|??|?????)\s*(.+)", u.text)
    if not m:
        return None
    target = re.sub(r"[????,?!]+$", "", m.group(1).strip()) or "??"
    return Reply(
        speech=f"??,?????{target}????",
        action={"type": "CALL", "target": target},  # ?????????
    )


@skill("??", priority=20)
def navigate(u: Utterance) -> Optional[Reply]:
    m = re.search(r"(?:????|?|???|?????)\s*(.+)", u.text)
    if not m:
        return None
    dest = re.sub(r"(???|???)$", "", m.group(1).strip())
    dest = re.sub(r"[????,?!]+$", "", dest)
    if not dest:
        return None
    return Reply(
        speech=f"???????{dest},???????",
        # ?? DeepLink,??????,?????
        action={"type": "OPEN_URI",
                "uri": f"androidamap://poi?sourceApplication=xiaoling&keywords={dest}&dev=0"},
    )


@skill("????", priority=30)
def play_media(u: Utterance) -> Optional[Reply]:
    m = re.search(r"(?:?|?|??|?????|?)\s*(.+)", u.text)
    if not m or not re.search(r"?|?|?|?|??|??|??|?", u.text):
        return None
    kw = re.sub(r"[????,?!]+$", "", m.group(1).strip())
    return Reply(
        speech=f"??,?????{kw}?",
        action={"type": "PLAY", "keyword": kw},
    )


@skill("??????", priority=40)
def health_remind(u: Utterance) -> Optional[Reply]:
    if not re.search(r"????.*(??|???|??|??|??)", u.text):
        return None
    m = re.search(r"(??|??|??)?\s*([0-9??????????]+?(?:?|[0-9]+??)?)", u.text)
    when = m.group(0).strip() if m else "??"
```

```
????????? V1.0   ? 69 ?
--------------------------------------------------------------------------------------------
    return Reply(
        speech=f"??,??{when}???,???",
        action={"type": "REMIND", "raw": u.text},
    )

// ===== server/translate.py =====
"""
?? ? ????(??? / ?? / ??)
???????????(0 ??),?????????(llm)?
??:???????????,??;??/????????
"""
from __future__ import annotations
import re

# ??????
LANGS = {"mandarin": "???", "english": "??", "cantonese": "??"}

# ??????:key=???,value={english, cantonese}
_PHRASES: dict[str, dict[str, str]] = {
    "??": {"english": "Hello", "cantonese": "??(nei5 hou2)"},
    "??": {"english": "Thank you", "cantonese": "??(do1 ze6)"},
    "??": {"english": "Goodbye", "cantonese": "??(baai1 baai3)"},
    "???": {"english": "How much is it?", "cantonese": "???(gei2 do1 cin2)"},
    "?????": {"english": "Where is the toilet?", "cantonese": "?????(ci3 so2 hai2 bin1 dou6)"},
    "????": {"english": "I don’t feel well", "cantonese": "????(ngo5 m4 syu1 fuk6)"},
    "???": {"english": "Please help me", "cantonese": "???(bong1 bong1 ngo5)"},
    "?????": {"english": "I need to go to the hospital", "cantonese": "?????(ngo5 jiu3 heoi3 ji1 jyun2)
    "????": {"english": "Call an ambulance", "cantonese": "????(giu3 gau3 wu6 ce1)"},
    "????": {"english": "I don’t understand", "cantonese": "????(ngo5 teng1 m4 ming4)"},
    "?????": {"english": "Please speak slower", "cantonese": "????(cing2 gong2 maan6 di1)"},
    "????": {"english": "Have you eaten?", "cantonese": "????(sik6 zo2 faan6 mei6)"},
    "???": {"english": "Good morning", "cantonese": "??(zou2 san4)"},
    "??": {"english": "Good night", "cantonese": "??(maan5 on1)"},
    "???": {"english": "Take care", "cantonese": "??(bou2 zung6)"},
    "????": {"english": "What time is it now?", "cantonese": "????(ji4 gaa1 gei2 dim2)"},
    "???": {"english": "How do I get there?", "cantonese": "??(dim2 heoi3)"},
    "???": {"english": "That’s too expensive", "cantonese": "???(taai3 gwai3 laa1)"},
    "???": {"english": "I love you", "cantonese": "???(ngo5 oi3 nei5)"},
    "????": {"english": "Wish you good health", "cantonese": "??????(zuk1 nei5 san1 tai2 gin6 hong1)"},
}


def parse_translate(text: str) -> tuple[str, str] | None:
    """
    ????????????? (?????, ????key) ? None?
    ?:’????? ??’ / ’?????? ??’ / ’??????????’
    """
    t = text.strip()
    if not re.search(r"??|???|?.{0,3}?|??|??", t):
        return None
```

```
?????????? V1.0   ? 70 ?
--------------------------------------------------------------------------------------------
    lang = "english"
    if re.search(r"??|???|??", t):
        lang = "cantonese"
    elif re.search(r"??|??", t):
        lang = "english"
    elif re.search(r"???|??|??", t):
        lang = "mandarin"
    # ???????:???????
    content = re.sub(r"(?)?(??)?(?)?|??(?|?)?|?.{0,3}?(???|?)?|???|??|??|??|??|???|??|???|??|??|[,??
    return (content or t, lang)


def translate_phrase(content: str, lang: str) -> str | None:
    """????????;??????,????? None(?????)?"""
    c = content.strip().rstrip("?.!?,")
    if lang == "mandarin":
        return c   # ??????,??(?????????? ASR ?)
    entry = _PHRASES.get(c)
    if entry:
        return entry.get(lang)
    # ????:??????
    for k, v in _PHRASES.items():
        if k in c or c in k:
            return v.get(lang)
    return None
```